

Golden Thread Quest: System Design

How the local-first Golden Thread Quest application is designed and why, what is finished, and what remains. Written for a developer or technical stakeholder new to the system.

Version	1.0.0
Date	2026-09-24
Describes	main at 16a0b03, the merge of pull request 8 (round 11)
Audit status from	chore/round-12-audit at 7a8cf7b (round 12 findings, fixes not merged)
Source	docs/design/ (Markdown is the source of truth)

CONTENTS

1. About this document
2. Part 1. Purpose and users
 - 1.1 The problem
 - 1.2 Who uses it
 - 1.3 What the application is, and what it is not
 - 1.4 The system in context
 - 1.5 The ownership boundary
 - 1.6 How progress stays honest
3. Part 2. Architecture
 - 2.1 Two halves: a generator and a small service
 - 2.2 The components
 - 2.3 Content loading and JSON Schema validation
 - 2.4 View models and Jinja2 rendering
 - 2.5 The loopback action service
 - 2.6 The validator runner
 - 2.7 The progress store and locks
 - 2.8 The build pipeline and last-known-good output
 - 2.9 The update and migration path
4. Part 3. Data, identifiers and state
 - 3.1 The repository folders and who owns them
 - 3.2 Records and what each one holds
 - 3.3 Stable identifiers
 - 3.4 Versions and hashes
 - 3.5 The attempt state machine
5. Part 4. The system in motion
 - 4.1 How to read these diagrams
 - 4.2 A participant starts a quest and assembles evidence
 - 4.3 Running a validator
 - 4.4 Submitting for review
 - 4.5 The reviewer's decision
 - 4.6 The build and render pipeline
 - 4.7 Taking an upstream curriculum update
 - 4.8 One browser action through the loopback service

6. Part 5. The security model

- 5.1 Threats
- 5.2 The loopback boundary
- 5.3 No arbitrary command, no arbitrary path
- 5.4 Allowlisted writes
- 5.5 Untrusted content in the pages
- 5.6 Validator containment
- 5.7 Secret scan and redaction
- 5.8 Review integrity
- 5.9 Hostile repository files
- 5.10 External systems and privacy

7. Part 6. Key decisions

- 6.1 Product shape
- 6.2 Content and contracts
- 6.3 Generation and runtime
- 6.4 Safety and integrity
- 6.5 Concurrency and failure
- 6.6 Patterns across the decisions

8. Part 7. Status and remaining work

- 7.1 How the work was staged
- 7.2 Why Stage 10 is still open: DH7
- 7.3 Audit history
- 7.4 Round 12: open findings
- 7.5 Known limitations
- 7.6 Deferred work
- 7.7 What done looks like

9. Part 8. Glossary

About this document

About this document

This is the teaching design document for the Golden Thread Quest application. It is written for a developer or technical stakeholder who has never seen the system, and it has two jobs:

1. explain what the system is for and how it is designed, in order, with the reason for each design choice;
2. say what is finished, what is open, and what is deliberately left for later.

It describes the code at the commit above. Where the code and the specification documents (`docs/ARCHITECTURE.md`, `docs/CONTENT-MODEL.md` and others) disagree, this document follows the code and says so at the point of disagreement. Where an open audit finding shows the code falling short of the design, the finding is named beside the claim.

The Markdown files in this folder are the source of truth. The HTML and PDF are rendered from them and carry the same version in their file names and on their title page. `REGENERATING.md` explains the version rule and how to rebuild both.

Conventions

- Every diagram is Mermaid, drawn from the code, with a numbered walkthrough beside it. GitHub renders the Mermaid blocks; the HTML carries them as inline SVG.
- `ADR-nnn` refers to `docs/DECISIONS.md`. Round 12 finding IDs (C1, E1, T1 and so on) refer to `docs/audits/round-12-independent-audit.md` on `chore/round-12-audit`. `Dn` refers to the deferred-work register in `docs/IMPLEMENTATION-PLAN.md`.
- Commands are run from the repository root. `quest-app` needs the virtual environment active: `make setup`, then `source .venv/bin/activate`.
- Terms are defined at first use and collected in the glossary.

Part 1. Purpose and users

This part explains the problem the application solves, who uses it, and the one boundary that shapes every later design choice: who owns which files.

1.1 The problem

The Golden Thread Quest trains an **AI Context Engineer**: a person who owns the two ends of software delivery that agentic tools touch most.

- The **intent bookend**: capturing conversations, decisions and requirements, structuring them as local context, and turning them into actionable work items.
- The **validation bookend**: turning that intent into tests, running them, keeping evidence, and saying what risk remains.

The "golden thread" is the traceable line between the two:

Conversation → requirement → work item → implementation → test → evidence → outcome

Training for this role has a specific difficulty. The skills are practical, so a quiz proves little. The work happens in real repositories with real tools, so a hosted course cannot see it. And the most important question a mentor asks, "did this person actually do it, and does it hold up?", needs a human judgment backed by evidence, not a checkbox.

The application answers that difficulty with three commitments:

1. **The curriculum is content.** Each unit of work is a **quest**: one Markdown file with YAML front matter that states the mission, acceptance criteria and required proof. Adding a quest needs no code change.
2. **The participant's work is ordinary files in their own Git repository.** The application reads those files, checks them and presents them. It does not own them and does not hold them in a database.
3. **Progress stays honest.** What a participant claims, what an automated check establishes and what a reviewer approves are different states with different authorities. Only a reviewer's approval produces *verified* completion.

1.2 Who uses it

Role	Who they are	What they need from the application
Participant	A business analyst, tester, Scrum facilitator, QA professional or developer learning agentic workflows, usually working alone and at their own pace	Know the next useful quest, understand what success means without hidden knowledge, build real artifacts, assemble evidence, resume after an interruption, and see the difference between "I finished" and "a reviewer confirmed it"
Reviewer (also called <i>Questmaster</i>)	A facilitator, mentor or peer who evaluates evidence	See what the quest asked for, what was submitted and what the checks found; reproduce it; approve, or ask for changes with reasons; never handle the participant's secrets
Curriculum maintainer (curriculum author)	The program owner who writes quests, regions, badges and validators	Add or change curriculum through content files alone, and get an actionable error when content is wrong

A fourth actor is not a person: the **upstream repository**, the canonical copy of the program from which participants take curriculum updates (ADR-002).

1.3 What the application is, and what it is not

It **is** a local-first, repository-native training application with two halves:

- a **generator** that turns validated content and participant state into a static HTML site (ADR-006, ADR-008), and
- a small **loopback service**, an HTTP server bound to `127.0.0.1`, that performs the few things a static page cannot: record a state change, run a registered check, rebuild (ADR-008, ADR-022). A command-line surface, `quest-app action`, performs the same actions through the same code for places with no browser, such as an agent sandbox.

It **is not** a learning-management system, a hosted multi-user service, a central database, a leaderboard, or a tool that writes to Jira, Trello or GitHub on the participant's behalf. Release one performs no external write at all. These exclusions come from `PLANNING-STATUS.md` and are enforced in code, not only stated.

1.4 The system in context

Walkthrough. Read the figure from the top.

1. The **curriculum maintainer** commits quests, regions, badges, tracks and validator code to the upstream repository. That content flows into each participant's fork by an ordinary Git merge that the participant runs.
2. The **participant** works in their fork. Their files live under `participant/`. They drive the application through a browser pointed at the loopback service, or through the `quest-app` command line.
3. The **reviewer** reads the participant's evidence, usually from a pull request or a checkout of the participant's repository, and records a decision through the same application. The decision is a file in the participant's evidence directory.
4. The application reads authored content and participant files, validates both, and writes generated HTML under `generated/`. It writes participant files only through a short allowlist of actions.
5. The application makes **no outbound network request of its own**. External systems (Jira, Trello, GitHub) are things the participant's own scripts talk to while doing a quest; the application never does.

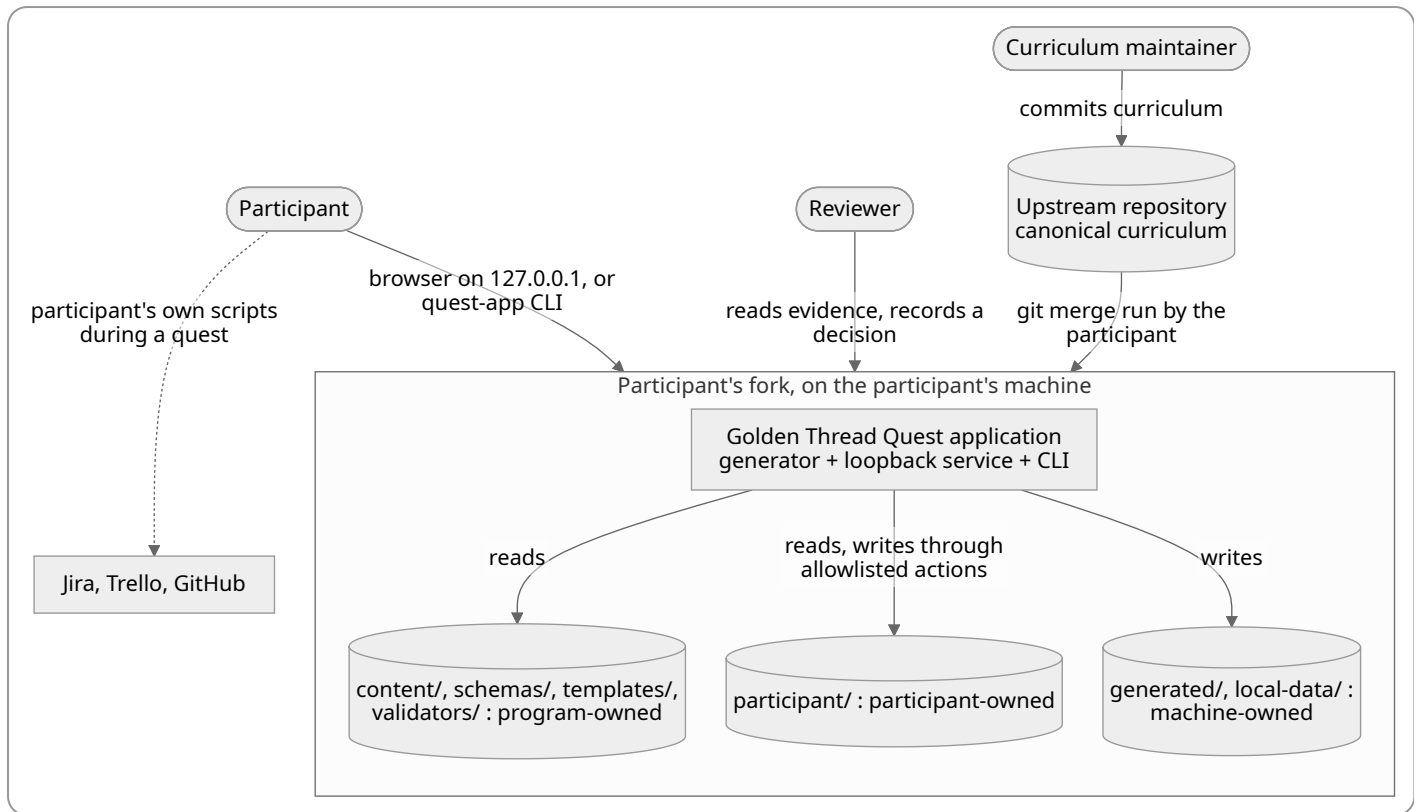


Figure 1. System context. Solid arrows are what the application does; the dotted arrow is work the participant does outside it.

1.5 The ownership boundary

Every file in the repository belongs to exactly one of three owners (ADR-003). This is the most important idea in the design, because the rest of the architecture exists to keep the three apart.

Zone	Folders	Owner	Who may change it	What happens on an upstream update
Program-owned	content/, schemas/, templates/, assets/, quest_app/, validators/, most of docs/	The program (curriculum maintainer and application developers)	Upstream, by commit	Merged in
Participant-owned	participant/	The participant	The participant in any editor, and the application through a narrow set of actions	Never replaced
Machine-owned	generated/, local-data/	Nobody: disposable	The application	Irrelevant: Git ignores them and a build recreates them

Three consequences follow, and each appears again later:

- **Updates cannot destroy work.** Because the participant's files sit in their own folder, taking new curriculum is an ordinary merge that touches only program-owned folders (Part 4, flow 4.7).

- **Generated output is never the truth.** A deleted or corrupt `generated/` directory is fixed by running a build (ADR-015). Canonical truth is content, participant files and review records.
- **The participant's files are untrusted input.** The participant can edit `participant/progress.yaml` by hand. The application therefore re-derives anything the participant is not allowed to decide, above all `verified`, from the review record rather than believing the file (ADR-011, Part 3).

Part 3 draws the folders in detail. Part 5 explains how writes are confined to the participant zone.

1.6 How progress stays honest

The application separates four kinds of statement about a quest, each with its own authority:

Statement	Authority	Example
"I have started" / "My evidence is ready"	The participant	<code>in_progress</code> , <code>evidence_ready</code>
"The automated checks pass"	A registered validator, requested by the participant	<code>locally_validated</code>
"Please review this"	The participant	<code>submitted</code>
"This is verified" / "This needs changes"	A reviewer	<code>verified</code> , <code>needs_changes</code>

Two XP totals follow from this and are never added together:

- **Claimed XP** counts quests from `evidence_ready` onward. It is the participant's own record.
- **Verified XP** counts only quests with an approved review. Nothing a participant does and nothing a validator returns can produce it (ADR-011).

The **proof hierarchy** says what counts as good evidence, strongest first: a repository artifact, reproducible instructions, automated execution evidence, a peer demonstration, and reviewer approval. Screenshots are supporting evidence only (ADR-013).

Part 2. Architecture

This part names the components, explains what each one is responsible for, and says why it is shaped that way. Part 4 then walks through the same components in motion.

2.1 Two halves: a generator and a small service

The application is a **hybrid** (ADR-008):

- **Static generation.** Python loads the content and the participant's state, validates both, computes every derived fact (which quests are locked, what XP is claimed or verified, which badges are earned), and renders HTML pages with Jinja2 into `generated/`. The pages are ordinary files: they open from disk with no server.
- **A loopback action service.** A small HTTP server on `127.0.0.1` serves those pages and accepts a short list of actions: start a quest, mark evidence ready, run a validator, submit, record a review, rebuild. After each action it rebuilds the site so the pages show the new state.

Why this split. Pure static HTML cannot create files, run checks or read Git, so some process has to. A full dynamic web application would duplicate state between a server and a browser and widen the attack surface. The hybrid keeps everything that is read as static, deterministic output and everything that writes behind one narrow, checkable door.

A third surface, the **command line** (`quest-app`), calls the same action code directly. It exists because the primary target surface, an agent sandbox such as the Cowork app, has a filesystem but cannot reach a loopback port from the user's browser.

2.2 The components

Walkthrough. The figure groups modules by responsibility. Read it top to bottom.

1. **Surfaces.** Three entry points: the CLI (`cli.py`), the loopback service (`serve.py`), and the static pages themselves. The CLI and the service both call the **action layer**; neither holds any rule of its own about what a participant may do (ADR-033).
2. **Action layer.** `actions.py` holds `ActionRunner`, the single allowlist of mutating actions and every guard: prerequisites, confirmation, secret scan, qualifying validation. `state_machine.py` holds the transition table and the confirmation texts.
3. **Loading and validation.** `pipeline.load_world` is the one way to obtain a consistent picture of the world: it calls `content_loader.py` (discover, parse, schema-validate, normalize), `semantics.py` (cross-document rules), `validator_registry.py` (validator references), and `progress.py` (participant state, read but never believed).
4. **Derivation and presentation.** `progress_calc.py` computes states, XP and badges; `recommend.py` picks the next quest; `view_models.py` turns all of it into display-ready objects; `build.py` renders templates, writes indexes and publishes atomically.
5. **Writing and running.** `store.py` writes participant files atomically under a lock; `review.py` writes submission and review records; `evidence.py` detects proof, scans for secrets and hashes evidence; `validator_runner.py` runs a validator in a child process (`validator_child.py`).
6. **Repository tools.** `git_status.py` runs five read-only Git commands; `update.py` and `migrations.py` handle upstream updates.

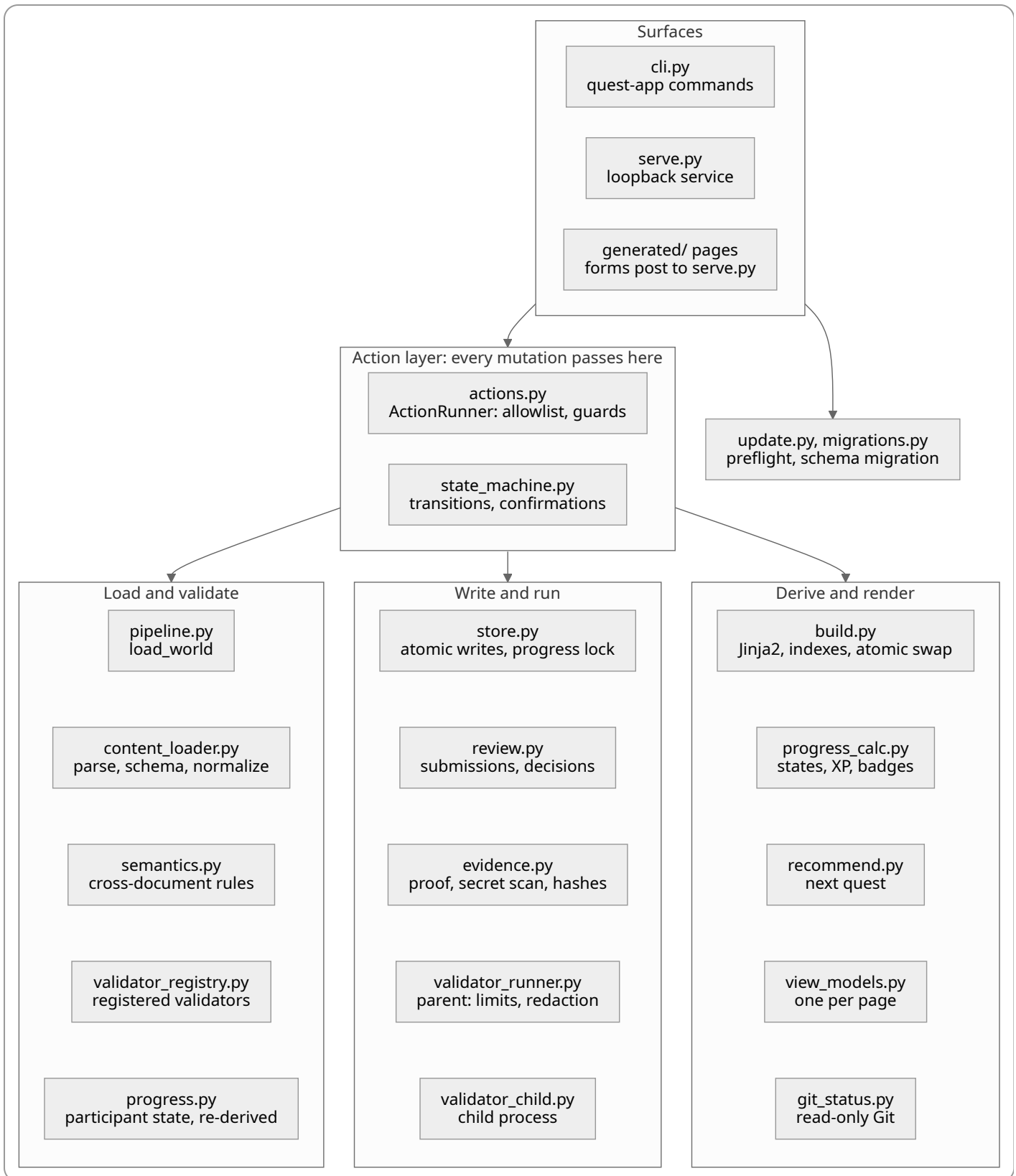


Figure 2. Components as implemented in `quest_app/`. Arrows run between groups: the action layer calls into all three lower groups, and `build.py` writes the pages the surfaces serve. `docs/ARCHITECTURE.md` names a single `progress_manager.py`; the code splits that role across `progress.py` (read), `store.py` (write), `progress_calc.py` (derive), `state_machine.py` and `actions.py`.

2.3 Content loading and JSON Schema validation

What happens. `content_loader.load_content` runs the same four steps every time:

1. **Discover** files under `content/` (quests are `content/quests/<region>/<name>.md`; regions, badges and tracks are YAML; `content/site.yaml` holds site identity). Only regular files inside the content tree are read, so a symbolic link to a device or a FIFO cannot hang the loader.
2. **Parse** YAML with a strict safe loader (`yaml_loader.StrictSafeLoader`): safe constructors only (ADR-025), duplicate keys refused (ADR-029), deep nesting reported as an ordinary problem.
3. **Validate** each document against its JSON Schema in `schemas/` using `jsonschema` Draft 2020-12 (ADR-005, ADR-020). There are ten schemas: quest, region, badge, track, site, progress, submission, review, validation-result and validator-registry.
4. **Normalize** each valid document into a frozen dataclass (`models.py`). Nothing downstream sees a raw dictionary. Quest Markdown is rendered with `markdown-it-py`, raw HTML disabled, then sanitized with `nh3` (ADR-021); the result is carried in a field named `safe_rendered_html` so it cannot be confused with plain text.

`semantics.validate_bundle` then applies the rules a schema cannot express, because they span documents: duplicate IDs, unknown region, prerequisite, badge or related-quest references, prerequisite cycles, a quest whose proof names a validator it does not declare, empty regions, unreachable badges, and more. Some are errors that stop the build; others, such as an unusual XP value for the chosen level, are warnings.

Why schemas are the contract. A JSON Schema is language-neutral, editor-friendly and readable by a maintainer who does not read Python. A second model layer (for example Pydantic) would restate the same rules in a place that can drift from the file the maintainer reads (ADR-020).

Why errors are shaped as they are. Every failure is a `ContentProblem` naming the file, the stable ID when known, the field path, the rule in words, a redacted received value, and a suggestion (for a mistyped ID, the closest real one). The person fixing it is a curriculum author in a text editor, not a developer reading a traceback.

Acceptance criteria. A quest's criteria are not duplicated into front matter. They are read from the ordered list under the body's `## Acceptance criteria` heading, from the Markdown token stream rather than by line matching (ADR-027), and each gets the positional ID `ac-<n>` plus a hash of its text (ADR-016, amended by ADR-026). A reviewer's finding can then point at `ac-3`, and the page and the finding agree on what `ac-3` means.

2.4 View models and Jinja2 rendering

What happens. `build.build_site` takes a `LoadedWorld` and:

1. computes per-quest state with `progress_calc.compute_states` (Part 3 explains the rule), region progress, claimed and verified totals, badge progress, and recommendations;
2. builds one view model per page in `view_models.py` (for example the quest summary, the evidence workspace, the reviewer view), each a display-ready object;
3. renders the thirteen page templates in `templates/pages/` with Jinja2, autoescaping on and `StrictUndefined` so a missing field is an error rather than an empty string;
4. writes search, tag, region, relationship and state indexes as JSON under `generated/indexes/`, plus `build-manifest.json` with the application version and the content hashes;
5. publishes the whole site atomically (Section 2.8).

The eleven screens specified in `docs/ui/SCREEN-SPECS.md` (U01 Home to U11 Content Author Error) map to thirteen templates: the extra two are the tag page and the evidence index.

Why view models. Templates contain no quest-specific content and no logic that decides anything (`docs/VIEW-MODEL-CONTRACT.md`). They do not open files, calculate progress, resolve prerequisites or decide authority. That keeps one place, Python, responsible for every rule, and lets a test assert that adding a quest leaves `templates/`, `assets/` and `quest_app/` byte-identical.

Why deterministic. Traversals are sorted and JSON is written with sorted keys, so a diff of `generated/` means content changed. The one varying value is the build time on each page. Set `SOURCE_DATE_EPOCH` and two builds of the same inputs are byte-identical (ADR-040).

2.5 The loopback action service

What it is. `serve.py` runs `http.server.ThreadingHTTPServer` from the Python standard library with an explicit route table (ADR-022). It serves `generated/` and exposes:

Route	Method	Purpose
<code>/api/health</code>	GET	Application and service status
<code>/api/git-status</code>	GET	A summary from the five read-only Git commands
<code>/api/action</code>	POST, JSON body	Perform one allowlisted action
<code>/api/action/<action>/<quest-id>[<validator-id>]</code>	POST, HTML form	The same, from a page with no JavaScript
anything else	GET	A static file from <code>generated/</code> , with the request token substituted into HTML

Why the standard library. Every requirement on the service is a *restriction*: bind to loopback, require a token, check origin, cap the body, accept only allowlisted IDs. A framework adds dependencies and hides the exact request surface those restrictions apply to. One local user needs no concurrency model beyond threads.

The request token. At startup the service mints a random token (`secrets.token_urlsafe(32)`), keeps it in memory and never writes it to disk or prints it. Built pages contain a placeholder; the service substitutes the token as it serves each HTML page, so a form on a page this run served carries it. A page opened from disk, or served by another run, does not have it, and its forms are refused.

Which port is running. The service writes the port it bound to a file per port under `local-data/service-ports/` and removes it on shutdown (ADR-037). A CLI action or build probes those ports, and believes an answer only if it carries this application's response header and repository signature. It needs to know, because a rebuilt page says whether its controls work: pages built while a service runs have live forms; pages built without one say "Start the local service".

Part 5 lists every check a request meets. Part 4, flow 4.8, shows one request end to end.

2.6 The validator runner

What a validator is. A **validator** is a program-owned Python function that checks a participant's artifacts and returns structured findings. Each is registered by stable ID in `validators/registry.yaml` with an endpoint (`validators.<module>:run`), a working directory, read and write roots, an environment-variable allowlist, a timeout, an output limit, the quests it may run for, and a `network: denied` declaration. A quest names a validator by ID and can never supply a command, argument or path (ADR-014).

Release one registers three quest-facing validators (`validate-repository-foundation`, `validate-jira-read-assigned`, `validate-playwright-quality`) and two environment probes used by tests.

How a run is isolated. `validator_runner.run_validator`:

- starts a fresh interpreter as a child process with a fixed argument list and no shell, in its own session and process group (`start_new_session=True`);
- builds the child's environment from a minimal `PATH` plus the registry's allowlist, never inheriting the parent's environment, so an exported credential does not leak into a check;
- puts the repository first on the child's import path, so a `json.py` in the participant's folder cannot replace a standard module or the registered code;
- gives the validator a `Workspace` object whose read and write methods resolve symbolic links and re-check that the path is inside the registered roots, and that exposes the evidence package of *the attempt under validation* (ADR-039);
- reads output as it arrives, stops a run that writes more than 1 MiB on a stream, kills the whole process group on timeout or exit, and redacts secret-like values before the result is stored or shown.

Outcomes. A run ends in one of six outcomes: `pass`, `fail`, `warning`, `environment_failure`, `inconclusive`, `interrupted`. Only `pass` and `warning` are **qualifying** results. Part 4, flow 4.3, shows how each outcome is reached.

A run is not a transition. Running a validator appends a result file and changes nothing else (ADR-017). The participant then asks for `locally_validated`, and that request succeeds only if every declared validator's latest run qualifies. A validator is therefore a source of facts, never an authority over the participant's state.

What this is not. The controls bound a validator through its `Workspace` and its process. They are not an operating-system sandbox: `network: denied` is a declaration nothing enforces, a validator calling `open()` directly bypasses the roots, and a grandchild that starts its own session survives cleanup (`docs/VALIDATOR-CONTRACT.md`; deferred item D9). Validators are program-owned code, reviewed like the rest of the application.

2.7 The progress store and locks

The store. `store.ProgressStore` is the only writer of `participant/progress.yaml`. Every write:

1. validates the new document against `schemas/progress.schema.json` *before* writing, so the application never writes a file it would later refuse to load;
2. writes to a temporary file in the same directory, flushes and `fsync`s it, then renames it over the original (`atomic_write_text`). A rename within one filesystem is atomic, so an interrupted write leaves the old file intact;
3. appends a line to `participant/ACTIVITY.md` describing the change, so the participant can read everything the application did to their files.

Three locks. Each one protects a different race:

Lock	Where	Protects	Decision
Service lock	A <code>threading.Lock</code> in the service process	Two browser requests interleaving inside one service	<code>serve.py</code> ; context in ADR-034
Progress lock	<code>participant/.progress.lock</code> , an exclusive POSIX <code>flock</code> , held by <code>ActionRunner.perform</code> across load, decision and write	A CLI action and a service action (two processes) interleaving a read-modify-write	ADR-034
Build lock	<code>generated.lock</code> , an exclusive <code>flock</code> held for the whole build	Two builds sharing the fixed staging directory <code>generated.building</code>	ADR-035

The progress lock is held across the *load* as well as the write, because reading state another process is about to replace is the race. It is not held over the validator run, which is slow and only appends.

Why a lock failure never refuses the work. Where a lock cannot be taken (no `fcntl` module, an unopenable lock file, or a filesystem that answers `ENOLCK`, such as some network home directories), the work proceeds unlocked (ADR-036). The atomic rename still guarantees the file is never half-written, and refusing to record a participant's own work because of a lock file is worse than the race.

Why a failed rebuild does not fail the action. An action writes its change, then rebuilds. If the rebuild fails, the action is still reported as done, with an advisory that the site could not be rebuilt (ADR-038). The participant's record cannot be regenerated; the site can, by one command.

2.8 The build pipeline and last-known-good output

A build renders the entire site into a sibling directory, `generated.building`, and then publishes by renaming: the current `generated/` becomes `generated.previous`, the staged directory becomes `generated/`, and the previous copy is removed. Until that rename, a reader keeps seeing the old site. If content fails validation, nothing is rendered into `generated/` at all: the errors are written as a page to `local-data/build-errors/index.html` and the last good site stays in place.

`docs/ARCHITECTURE.md` lists "copy fingerprinted or versioned static assets" as a build step. The code copies `assets/` as-is; `routes.asset` accepts a fingerprint but no caller passes one.

Part 4, flow 4.6, draws the pipeline.

2.9 The update and migration path

A participant's fork has two remotes: `origin` (their fork) and `upstream` (the canonical repository). Taking new curriculum is a Git merge from `upstream`.

The application never runs the merge. `quest-app update` (also `make update-check`) is a **preflight**: it checks that the directory is a Git repository, that no merge is in progress, that the working tree is clean, that an `upstream` remote exists, and that the participant folder is present. It reports whether a schema migration would be needed and which attempts are on an older quest version, proposes a backup branch name, prints the exact commands, and stops. Every Git command `update.py` may run is read-only.

Migrations. `quest-app update --migrate` (also `make migrate`) moves `participant/progress.yaml` to the schema version this build reads. It runs under the progress lock, validates before and after, reloads the whole world after writing, and restores the original bytes if that reload fails. Two rules apply to every migration: an in-progress attempt's quest version is never advanced, and an unknown field is carried forward rather than dropped. Release one ships schema version 1, so the only step is the identity step; the machinery exists so the safety is designed before the first real migration.

Where this differs from the plan. `docs/ARCHITECTURE.md` describes a future helper that "may create a backup branch, merge or rebase, apply migrations". The code does none of the Git writes: it names the branch and prints the commands (ADR-032, `update.py`). Part 4, flow 4.7, draws the sequence.

Part 3. Data, identifiers and state

This part describes where each kind of record lives, how records refer to one another, and the state machine an attempt moves through.

3.1 The repository folders and who owns them

Walkthrough.

1. The top-left group is **program-owned**. The curriculum maintainer and the application developers change it upstream, and it arrives in a fork by merge.
2. The top-right group is **participant-owned**. It does not exist in a fresh clone: the first `start-quest` action creates `participant/progress.yaml`, `participant/ACTIVITY.md` and the first evidence package. After that the participant owns it, commits it, and may edit it.
3. The bottom group is **machine-owned** and ignored by Git. Deleting it loses nothing that a build cannot recreate, except the error page of the last failed build and the port files of running services.
4. The arrows show the only read and write paths. The application writes into `participant/` only through the store, the evidence code and the review code, and only inside the configured participant root after symbolic links are resolved (ADR-018).

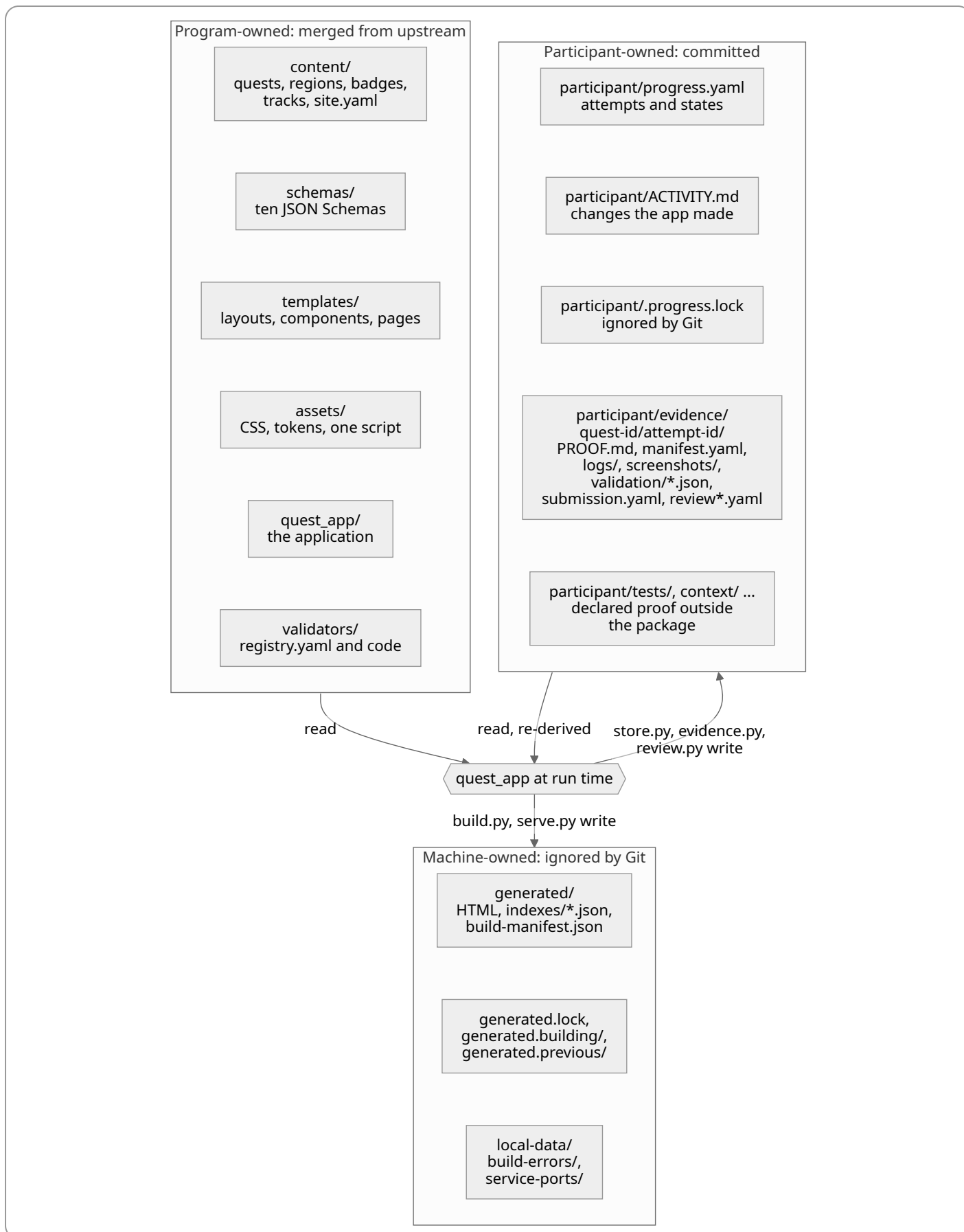


Figure 3. Repository folders by owner, from the code (`store.py` , `review.py` , `build.py` , `.gitignore`).

Where this differs from the plan. `docs/ARCHITECTURE.md` lists participant files the application does not create or read: `PROFILE.md` , `QUEST-PASSPORT.md` , `QUEST-LOG.md` , `skills/` , `scripts/` . The passport is a generated

page, not a participant file. The code creates only what the figure shows; a quest may ask the participant to create other folders, such as `participant/tests/`, as proof.

3.2 Records and what each one holds

Record	Format and location	Written by	Holds
Quest	Markdown with YAML front matter, <code>content/quests/<region>/<name>.md</code>	Maintainer	<code>id</code> , <code>version</code> , <code>title</code> , <code>summary</code> , <code>region</code> , <code>level</code> , <code>xp</code> , <code>estimated_minutes</code> , <code>tags</code> , <code>outcomes</code> , <code>proof</code> (required and optional), optional <code>prerequisites</code> , <code>validators</code> , <code>related_quests</code> , <code>risk</code> ; the body holds Mission, Scenario, Acceptance criteria, Required evidence, Safety constraints, Hints, Reflection
Region, badge, track, site	YAML under <code>content/</code>	Maintainer	Ordering and outcomes (region), criteria and authority (badge), quest selection (track), identity and navigation (site)
Validator registry	<code>validators/registry.yaml</code>	Program	Every program that may run, with its constraints (Section 2.6)
Progress	<code>participant/progress.yaml</code>	Application, and the participant by hand	Participant identity, selected track, focus tags, and one entry per attempt : <code>attempt_id</code> , <code>quest_id</code> , <code>quest_version</code> , <code>content_hash</code> , <code>state</code> , timestamps, <code>evidence_path</code> , <code>submission_id</code> , <code>review_id</code>
Evidence package	<code>participant/evidence/<quest-id>/<attempt-id>/</code>	Participant (content), application (template)	<code>PROOF.md</code> answering what was built, where, how to reproduce, what was checked, what remains, and a sensitive-values confirmation; <code>manifest.yaml</code> ; <code>logs/</code> , <code>screenshots/</code>
Validation result	JSON, <code>validation/<run-id>.json</code> inside the package	Validator runner	Run ID, validator ID and version, quest and attempt IDs, timestamps, outcome, checks, redacted output excerpt
Submission	<code>submission.yaml</code> inside the package	Application, on submit	Submission ID, quest version, attempt ID, content hash, evidence hash , proof_files digests, secret-scan result, validation run IDs, advisories
Review	<code>review.yaml</code> inside the package; superseded decisions archived as <code>review-<timestamp>.yaml</code>	Application, on the reviewer's decision	Review ID, quest ID and version, attempt ID, evidence hash, reviewer display name, decision, findings, verification statement (for approval), <code>proof_files</code>
Activity log	<code>participant/ACTIVITY.md</code>	Application	One line per change the application made
Generated site	<code>generated/</code>	Build	HTML pages, JSON indexes, build manifest. Never authoritative

Every participant-side record is validated against its schema before it is written and again when it is read.

3.3 Stable identifiers

Records refer to each other by **stable ID**, never by filename or title (ADR-012). An ID is lowercase ASCII letters, digits and hyphens, starts with a letter, is unique within its type, and is never reused for a different concept. Examples: `base-camp-repository-safety` (quest), `jira-jungle` (region), `validate-jira-read-assigned` (validator).

IDs the application mints:

ID	Shape	Example
Attempt	<code><first word of quest id>-attempt-<nnn></code>	<code>base-attempt-001</code>
Submission	<code>submission-<UTC timestamp>-<6 hex></code>	<code>submission-20260924142343-ca18d5</code>
Review	<code>review-<UTC timestamp>-<6 hex></code>	<code>review-20260924142355-1a2b3c</code>
Acceptance criterion	<code>ac-<n></code> , by position in the list	<code>ac-3</code>

Why IDs and not paths. Titles and filenames change. A reviewer's finding on `ac-3`, an attempt's `quest_id`, a badge's list of quests all survive a rename. URLs are derived from IDs by `routes.py`; content never stores an output path. Records are also connected by ID and never by modification time: a validator judges the package of the attempt it was given, not the newest file under `participant/evidence` (ADR-039).

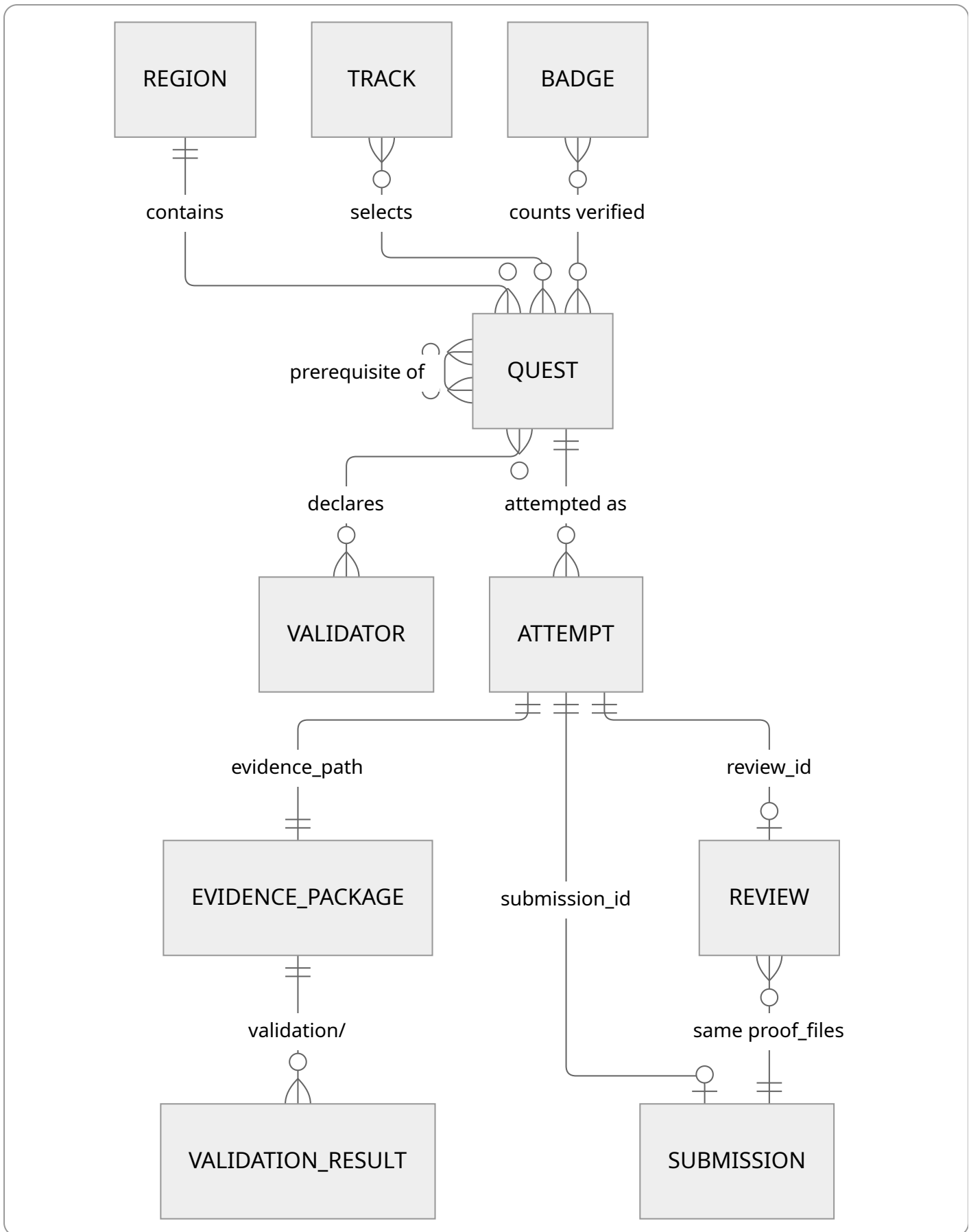


Figure 4. How records reference one another by stable ID. Program-owned records (region, track, badge, quest, validator) are on the left of each relation; participant-owned records (attempt and everything under it) on the right.

3.4 Versions and hashes

Three fingerprints let the application notice change without trusting anyone's word.

- **Quest version** (`version`, a positive integer) is declared by the author and must be raised when outcomes, criteria, proof, validators, XP, prerequisites or safety rules change. An attempt records the version it started on and keeps it through an update.
- **Content hash** (`content_hash`) is a SHA-256 over the quest's front matter and body. An attempt records it at start. A mismatch later is a **warning**, never an error: "the text changed since you started", or for a verified attempt "the approval may be stale" (ADR-028). It is a warning because the hash also changes on a typo fix, and a typo fix must not revoke someone's approved work.
- **Evidence hash** (`evidence_hash`) is a SHA-256 over the evidence package, excluding `validation/`, `submission.yaml` and `review*.yaml` (ADR-031). It answers "has the participant's work changed?", so the application's own bookkeeping stays out of it. Because most quests declare proof outside the package, submissions and reviews also record **proof_files**: a digest per declared proof path outside the package, `missing` when nothing is there and `unresolvable` when the path leads outside `participant/` (ADR-031, amended in round 11).

The submission records both fingerprints. The reviewer's approval is refused if either has changed since submission unless the reviewer acknowledges the change, and after approval the loader warns if the package or any proof file changes.

3.5 The attempt state machine

Terms. A **quest state** is what the pages show for a quest. Two quest states are *computed* and never stored: **locked** (a prerequisite is not yet verified) and **available** (every prerequisite is verified, no attempt exists). The other six are **attempt states**, stored in `progress.yaml`: `in_progress`, `evidence_ready`, `locally_validated`, `submitted`, `needs_changes`, `verified`. A prerequisite counts as met only when it is *verified*, not merely attempted.

Walkthrough.

1. `start-quest` creates the attempt and its evidence package, and moves the quest from available to `in_progress`. It is refused on a locked quest, and it needs a confirmation.
2. `mark-evidence-ready` is the participant's assertion that the proof is assembled. It is refused if the secret scan finds anything in the evidence package.
3. `mark-locally-validated` is refused unless every validator the quest declares has a qualifying latest result. A quest with no validators reaches it on the participant's word. Running a validator is **not** a transition.
4. `submit-for-review` works from `evidence_ready` or `locally_validated`. Unrun or failing validators become advisories on the submission, not blockers. A secret-scan hit, a link leading outside the package, or a missing package blocks it. It needs a confirmation.
5. `reopen-evidence` and `withdraw-submission` let the participant step back to `in_progress`.
6. The reviewer's `record-review` moves `submitted` to `verified` (approved) or to `needs_changes` (needs changes, or rejected). It needs a confirmation. `resume-quest` takes the participant from `needs_changes` back to `in_progress`.
7. `verified` has no outgoing transition.

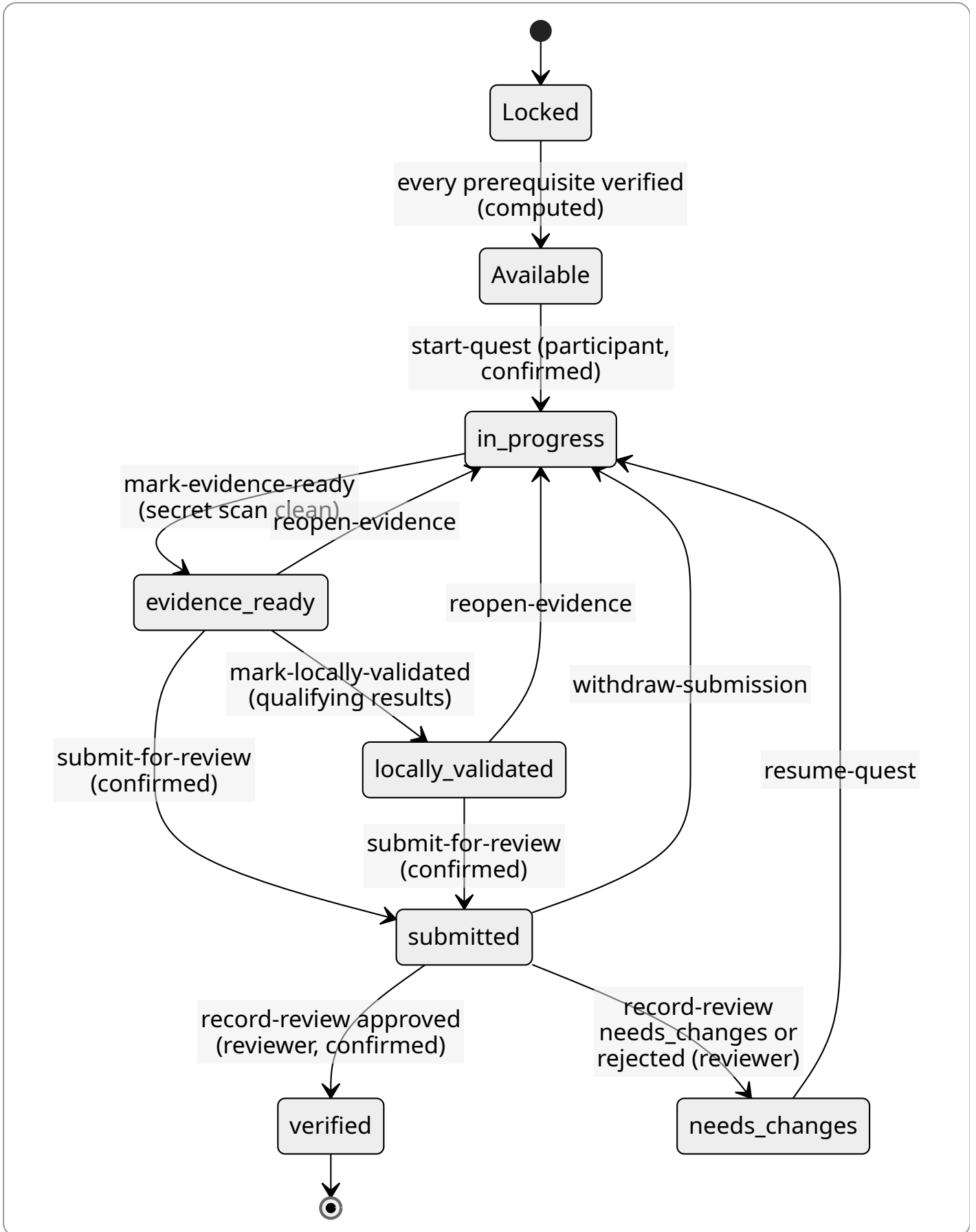


Figure 5. The attempt state machine as implemented in `quest_app/state_machine.py` (participant transitions) and `quest_app/review.py` (reviewer transitions). Locked and Available are computed in `progress_calc.py` and never written.

The rule that only a reviewer produces verified

Nothing in the transition table can produce `verified`. The one line of code that writes it is `review._apply_decision`, and it runs only after `record_decision` has checked that the attempt is `submitted`, that an approval carries a verification statement of at least twenty characters, and that the evidence has not changed since submission (or that the reviewer acknowledged the change).

Because `progress.yaml` is the participant's file, writing `state: verified` by hand is possible. The loader therefore re-derives `verified` on every load and refuses it, as the integrity error `progress.unverified_verified_state`, when the attempt names no review, the review is missing, belongs to another attempt or quest, is not an approval, has no verification statement, or approves a different quest version. The same applies to a hand-written `locally_validated` without a qualifying result for each declared validator (`progress.unvalidated_locally_validated_state`, ADR-017 amended in round 11).

Verified XP and badges are computed only from attempts that pass this check. Claimed XP counts `evidence_ready` and every later state.

Where the code and the specification disagree

`docs/ARCHITECTURE.md` draws a different machine. The code is authoritative; each difference is deliberate unless marked open.

Specification	Code	Reason
<code>EvidenceReady --> InProgress: Validation fails</code>	No such edge. A failing run leaves the state alone	ADR-017: a validator is not an authority over participant state
Submission only from <code>LocallyValidated</code>	Submission also from <code>evidence_ready</code> , with unrun or failing validators carried as advisories	<code>review.readiness_problems</code> ; the specification's own note allows submission with advisory failures if displayed
No step back from evidence-ready or submitted	<code>reopen-evidence</code> and <code>withdraw-submission</code>	Participant can correct work before a reviewer decides
Decision <code>rejected</code> (<code>CONTENT-MODEL.md</code>) with no state of its own	<code>rejected</code> maps to the <code>needs_changes</code> state	<code>review._apply_decision</code>
Spelling <code>needs-changes</code> (<code>CONTENT-MODEL.md</code>)	<code>needs_changes</code> everywhere	<code>models.Decision</code> , <code>models.AttemptState</code>
A <code>submitted</code> attempt always has a submission	A hand-edited <code>submitted</code> with no <code>submission.yaml</code> loads and can be approved, skipping the secret-scan gate	Open: round 12 finding E4 (Medium)
A legitimate <code>locally_validated</code> attempt survives an update	It becomes a load error when an update adds a validator to the quest, because the check reads the current quest's validators, not the attempt's version	Open: round 12 finding E5 (High)

Part 4. The system in motion

This part walks through the main flows. Each diagram is drawn from the code, and each flow names the functions that implement it so a reader can check.

4.1 How to read these diagrams

The activity diagrams are UML-style, drawn as Mermaid flowcharts:

- a **lane** (a labelled box) groups the steps one role or one process performs;
- a rounded box is an action; a diamond is a decision; a circle marks the start; a double circle marks the end;
- a box whose text begins **Refused** is a place the flow stops with a message and changes nothing.

Every mutating action in these flows passes through `ActionRunner.perform` (`quest_app/actions.py`), whichever surface asked. It performs three steps before the action itself: it refuses `start-quest`, `submit-for-review` and `record-review` without a confirmation; it takes the progress lock; and it loads and validates the whole world (`pipeline.load_world`). If content or participant state does not validate, every action is refused. The diagrams show these steps once, in flow 4.2, and abbreviate them after that.

The commands in this part are the command-line equivalent of the browser controls. Run them from the repository root with the virtual environment active (`source .venv/bin/activate`).

4.2 A participant starts a quest and assembles evidence

Walkthrough.

1. The participant chooses an available quest and confirms "Start this quest and create an evidence package in my repository".
2. The action layer checks the confirmation, takes the progress lock, and loads the world. It refuses a quest ID that is not in the loaded content, so no caller-supplied string ever reaches the filesystem as a path.
3. It refuses a locked quest: every prerequisite must be *verified* (ADR-033). This check is in the action layer, not only in the page, so the CLI cannot bypass it.
4. `store.start_attempt` creates `progress.yaml` if this is the participant's first action, refuses if the quest already has an attempt, mints the attempt ID, and creates the evidence package from a template: `PROOF.md` with its six headings, `manifest.yaml`, and empty `validation/`, `screenshots/` and `logs/` folders. Existing files are never overwritten.
5. It appends the attempt (quest version and content hash included), validates the new `progress.yaml` against its schema, writes it atomically, and appends a line to `ACTIVITY.md`.
6. The site is rebuilt. A failed rebuild becomes an advisory; the attempt is still recorded (ADR-038).
7. The participant does the work in their own editor and tools: artifacts under `participant/`, and `PROOF.md` filled in.
8. `mark-evidence-ready` runs the secret scan over the whole package. Any finding, any file the scan cannot read, and any symbolic link leading outside the package blocks it. The message names the file and line, never the value.

BASH

```
quest-app action start-quest --quest base-camp-repository-safety --confirm
quest-app action mark-evidence-ready --quest base-camp-repository-safety
```

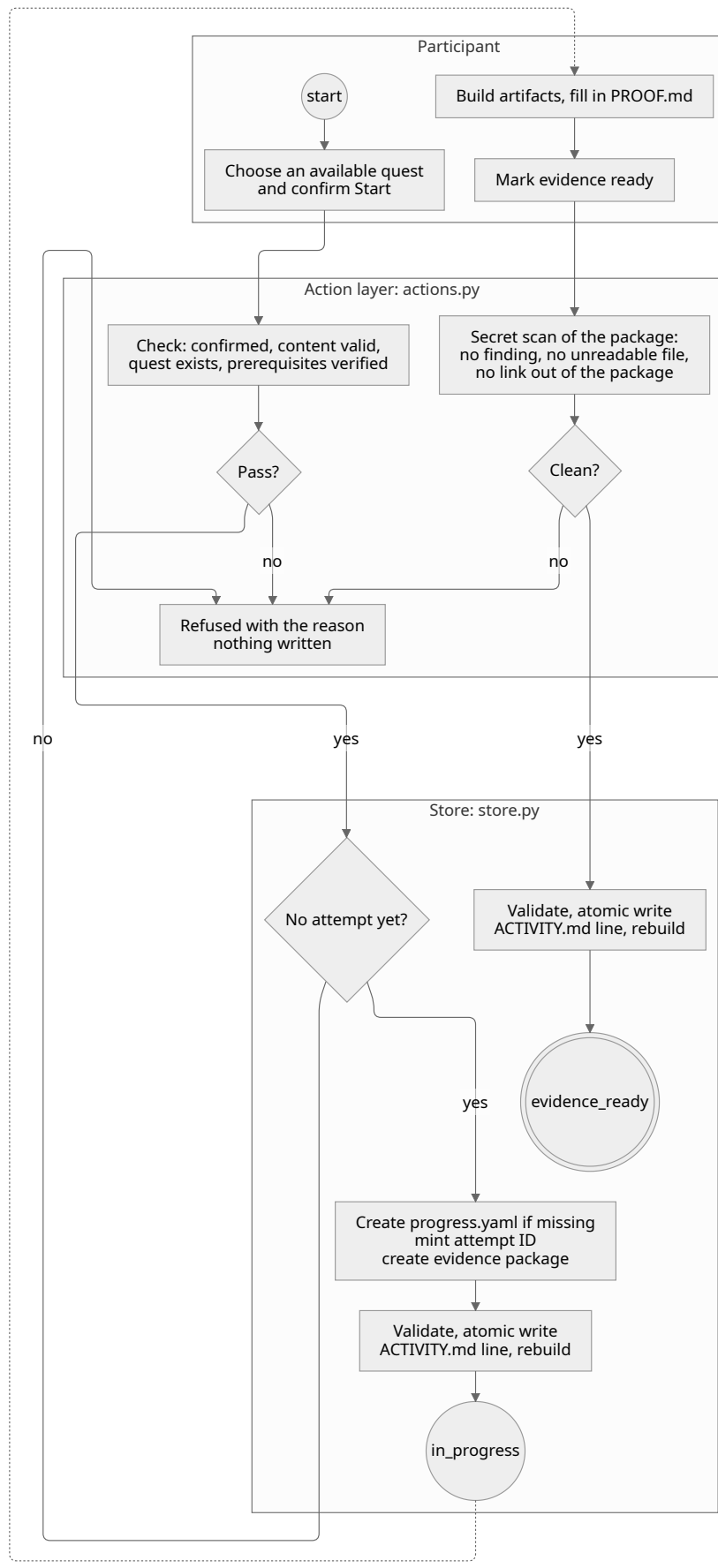


Figure 6. Starting a quest and marking evidence ready (`ActionRunner._perform`, `store.start_attempt`, `ActionRunner._require_clean_secret_scan`). Marking evidence ready needs no confirmation, but it takes the lock and loads the world like every action.

4.3 Running a validator

Walkthrough.

1. The participant runs one of the quest's validators. The action layer refuses a validator the quest does not declare, and a quest with no attempt.
2. The registry entry is looked up by ID. `run_validator` refuses a validator not registered for this quest, and binds parameters against their declared types and allowed values; there is deliberately no free-text parameter type.
3. The runner builds the child's environment from a minimal `PATH` plus the registry allowlist, sets the working directory from the registry, and starts `python -s -c <bootstrap>` in a new session and process group. The specification (roots, parameters, the attempt's evidence root) is sent on standard input. There is no shell and no argument built from input.
4. The child imports the entrypoint, but only from the `validators` package, and calls it with a `Workspace` and an output collector. The validator's own prints are moved to standard error so they cannot corrupt the result, which is written as JSON on the original standard output.
5. The parent reads both streams as they arrive. Four things can end the run: the child exits; the timeout passes; a stream exceeds 1 MiB; or the child exits and one second passes while grandchildren still hold a pipe. In every case the whole process group is killed.
6. The run is classified (next table). Every result carries at least one check, so an `interrupted` or `environment_failure` run still tells the participant what happened.
7. Output is truncated to the registered limit (at most 20,000 characters), redacted, and every free-text field of every check is redacted.
8. The result is validated against `validation-result.schema.json` and written atomically to `validation/<run-id>.json` in the attempt's package. The site is rebuilt. **The attempt's state does not change.**
9. Later, `mark-locally-validated` succeeds only if every declared validator's latest result is `pass` or `warning`.

Outcome	How it is reached (<code>validator_runner.py</code>)	Qualifies?
<code>interrupted</code>	The timeout passed	No
<code>environment_failure</code>	Output over 1 MiB; nonzero exit; empty output; unreadable JSON; the validator raised; the validator tried to leave its roots; or it reported an environment failure itself	No
<code>inconclusive</code>	No checks, every check skipped, or any check inconclusive (and none failed)	No
<code>fail</code>	Any check failed	No
<code>warning</code>	Any check warned, none failed or were inconclusive	Yes
<code>pass</code>	Every check passed	Yes

BASH

```
quest-app action run-validator --quest base-camp-repository-safety \  
  --validator validate-repository-foundation  
quest-app action mark-locally-validated --quest base-camp-repository-safety
```

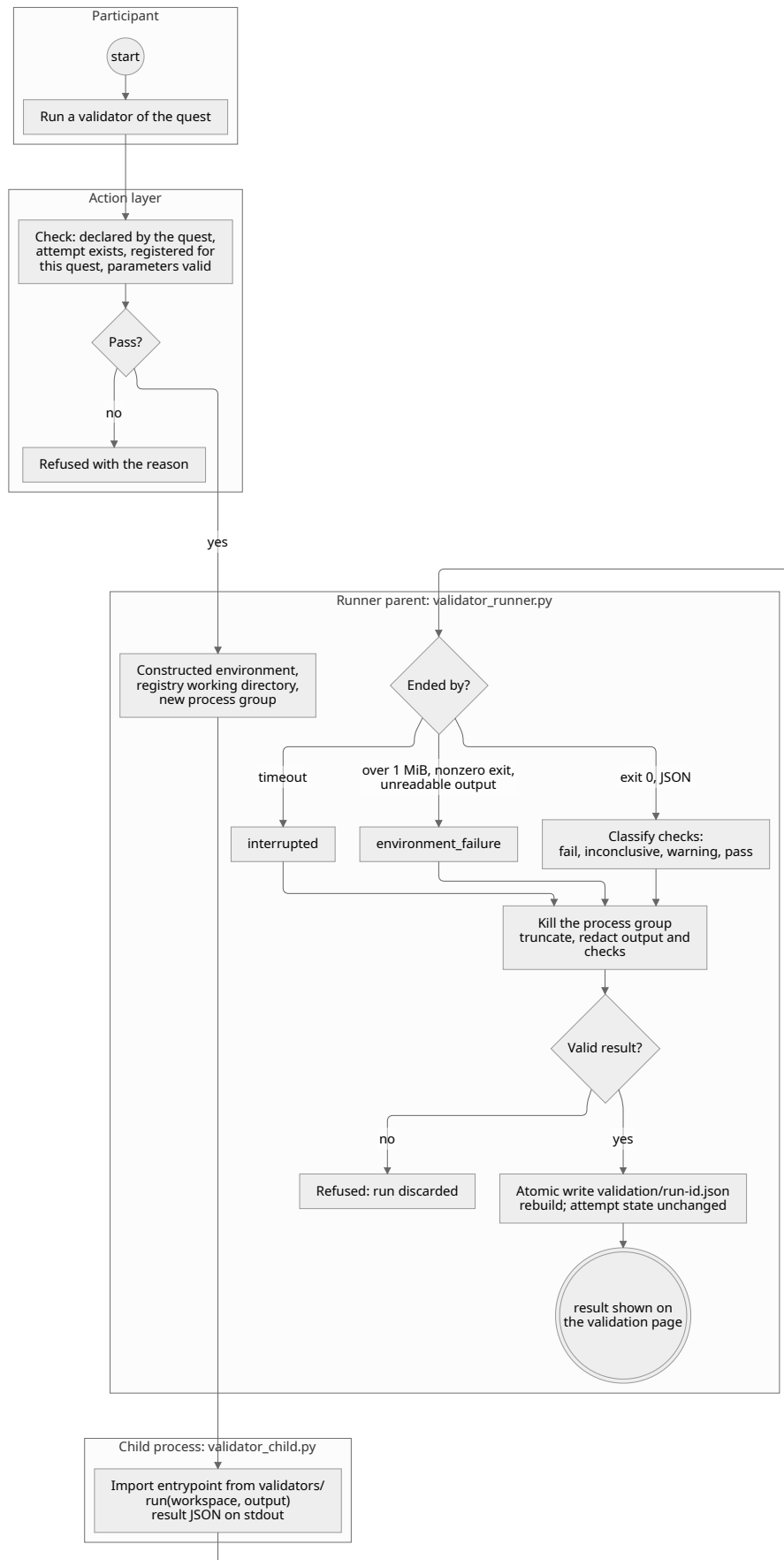


Figure 7. Running a validator (`ActionRunner._run_validator` , `run_validator` , `_collect` , `classify` , `validator_child.main` , `evidence.store_result`).

Open issues in this flow (round 12). An oversize `summary` , `evidence` or other check field is not capped, so the schema check refuses the whole run instead of recording it (E6, Medium). Redaction runs after truncation, so a

token cut at the boundary can be stored in clear (E7, Medium). A validator that finishes but leaves a non-daemon thread running is reported `interrupted` (E11, Low). A grandchild that starts its own session survives cleanup (E9, Low).

4.4 Submitting for review

Walkthrough.

1. The participant confirms "Submit this evidence for review. A reviewer will read it".
2. `review.readiness_problems` collects every problem at once, so the participant sees all of them together.
Blocking problems: the attempt is not `evidence_ready` or `locally_validated`; the secret scan found something; a link leads outside the package; the package is missing. **Advisory** problems: a declared validator has not been run, or its latest result does not qualify.
3. Any blocking problem refuses the submission.
4. Otherwise `create_submission` computes the evidence hash and the proof-file digests, writes `submission.yaml` (validated against its schema) into the package, moves the attempt to `submitted`, records the submission ID on the attempt, and appends an activity line.
5. The site is rebuilt, and the result carries the advisories and the Git commands for the next step.
6. **The application never pushes, opens a pull request or merges** (ADR-032). It prints the commands: create a branch, add the evidence and `progress.yaml`, commit, push, open a pull request. The participant runs them. Pushing is a claim, on the participant's behalf, that the work is ready, and that claim is theirs to make.

BASH

```
quest-app action submit-for-review --quest base-camp-repository-safety --  
confirm
```

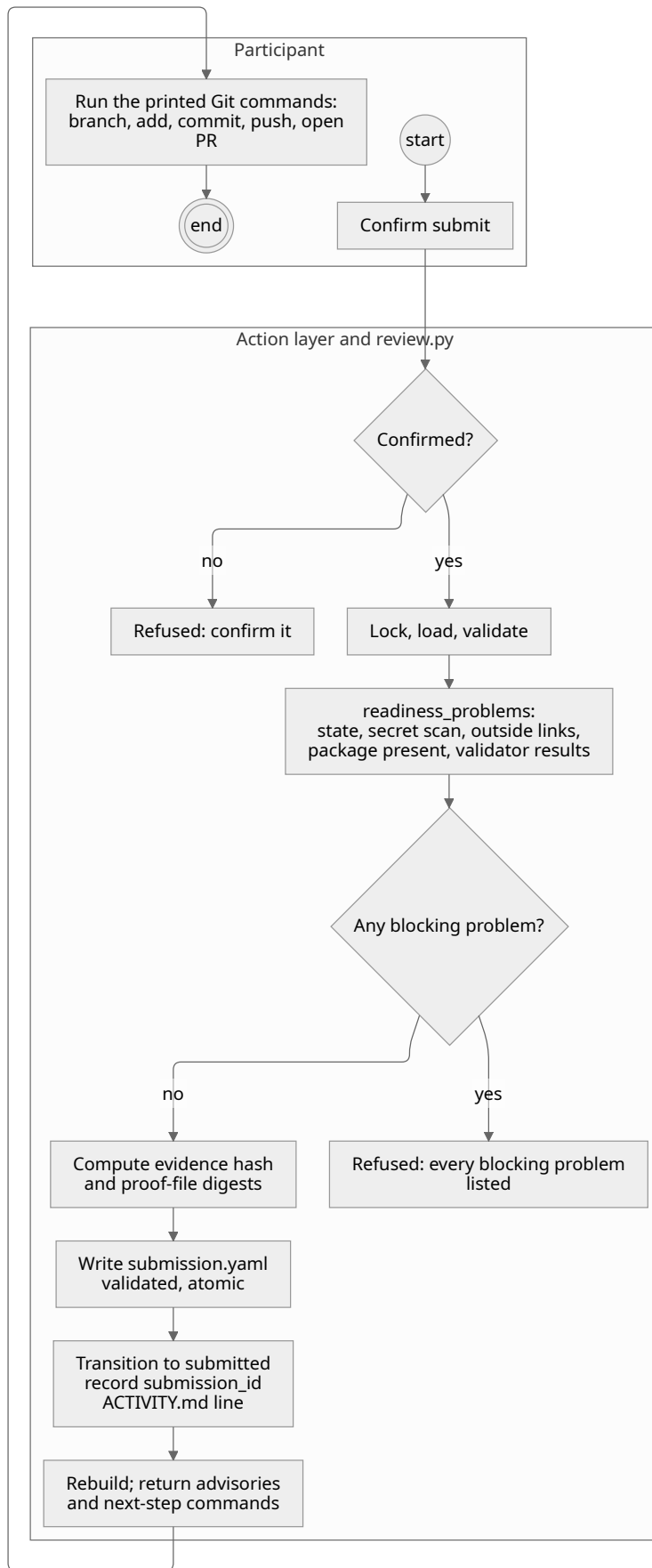


Figure 8. Submitting for review (`ActionRunner._submit`, `review.create_submission`, `review.readiness_problems`, `review.submission_instructions`).

4.5 The reviewer's decision

Walkthrough.

1. The reviewer reads the evidence: the reviewer page (`/review/<quest-id>/`) shows the quest and its version, the attempt, the submission, the validator results, the secret-scan result, and whether anything changed since submission, naming each changed path.
2. The reviewer chooses **approved**, **needs changes** or **rejected**, fills in the form, and confirms. The confirmation text depends on the decision; only the approval text says it produces verified XP.
3. `record_decision` refuses a decision that is not one of the three values, and an attempt that is not `submitted`.
4. An **approval** needs a verification statement of at least twenty characters saying what was checked and how. Then the **changed-since-submission gate**: the current evidence hash and each recorded proof-file digest are compared with `submission.yaml`. If anything differs, the approval is refused unless the reviewer acknowledges the change. Approving changed evidence would approve something the reviewer has not seen.
5. **Needs changes** and **rejected** need at least one finding with a severity, a summary and the evidence observed.
6. The review record is validated against its schema. An existing `review.yaml` is archived as `review-<timestamp>.yaml` rather than overwritten, so a change of mind stays visible. The new record is written atomically.
7. `_apply_decision` sets the attempt state: `verified` for an approval, `needs_changes` otherwise. This is the only code that writes `verified`.
8. The site is rebuilt. On the next load, `verified` is re-derived from this record (Part 3, Section 3.5), and verified XP and badges follow from it.
9. After `needs_changes`, the participant runs `resume-quest` and the cycle repeats.

BASH

```
quest-app action record-review --quest base-camp-repository-safety \  
  --decision approved \  
  --reviewer "Your Name" \  
  --statement "What you checked and how, in your own words." \  
  --confirm  
  
quest-app action record-review --quest base-camp-repository-safety \  
  --decision needs_changes \  
  --reviewer "Your Name" \  
  --finding "high:What is wrong:Where you saw it" \  
  --confirm
```

Add `--acknowledge-changed-evidence` to approve evidence that changed after submission; the browser form has the same checkbox.

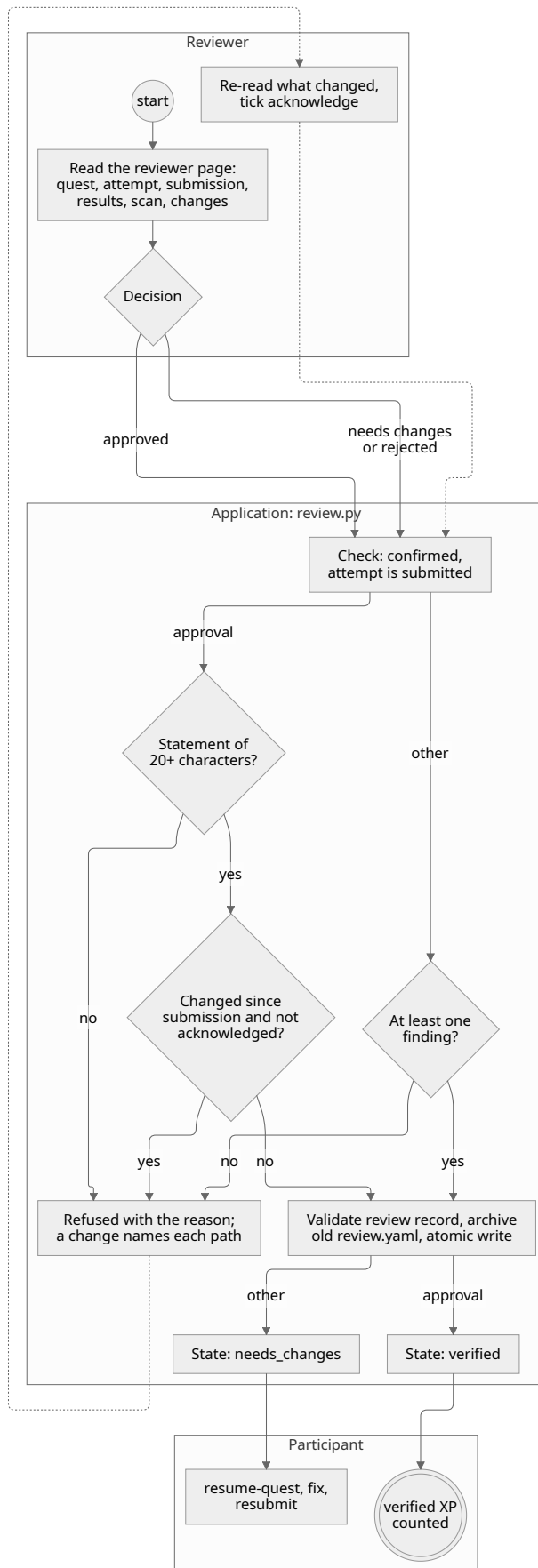


Figure 9. Recording a reviewer decision (`ActionRunner._record_review`, `review.record_decision`, `review.changes_since_submission`, `review._apply_decision`).

Provenance. A reviewer is identified by the display name in the record and by Git history. Records are not signed (ADR-030). The application refuses every internally inconsistent claim, but a participant with write access to their

own repository could write a record naming someone else. Programs that need more should review through pull requests so Git carries the identity. Release one also has no role separation: a participant can open the reviewer page for their own work (deferred item D11).

4.6 The build and render pipeline

Walkthrough.

1. `quest-app build` (also `make build`), `quest-app serve`, and every action's rebuild all call the same pipeline.
2. `load_world` loads content first: discover, parse strictly, validate against schemas, normalize, then apply cross-document rules and check validator references. Participant state is read only if content is valid, because an attempt cannot be judged against a curriculum that does not make sense. A missing `progress.yaml` is normal.
3. Participant state is read with bounded reads: each file must be an ordinary file under a size ceiling. `verified` and `locally_validated` claims are checked against their records.
4. **If anything is an error**, the build renders the error page (U11) to `local-data/build-errors/index.html`, prints its path, exits non-zero, and leaves the published site untouched.
5. **If valid**, `build_site` takes the build lock and renders into `generated.building`: derived states, totals, badges and recommendations; one view model per page; every page through Jinja2 with autoescaping; the JSON indexes and the manifest; a copy of `assets/`.
6. The publish is two renames. Only then does a reader see the new site.

BASH

```
make validate-content
make build
```

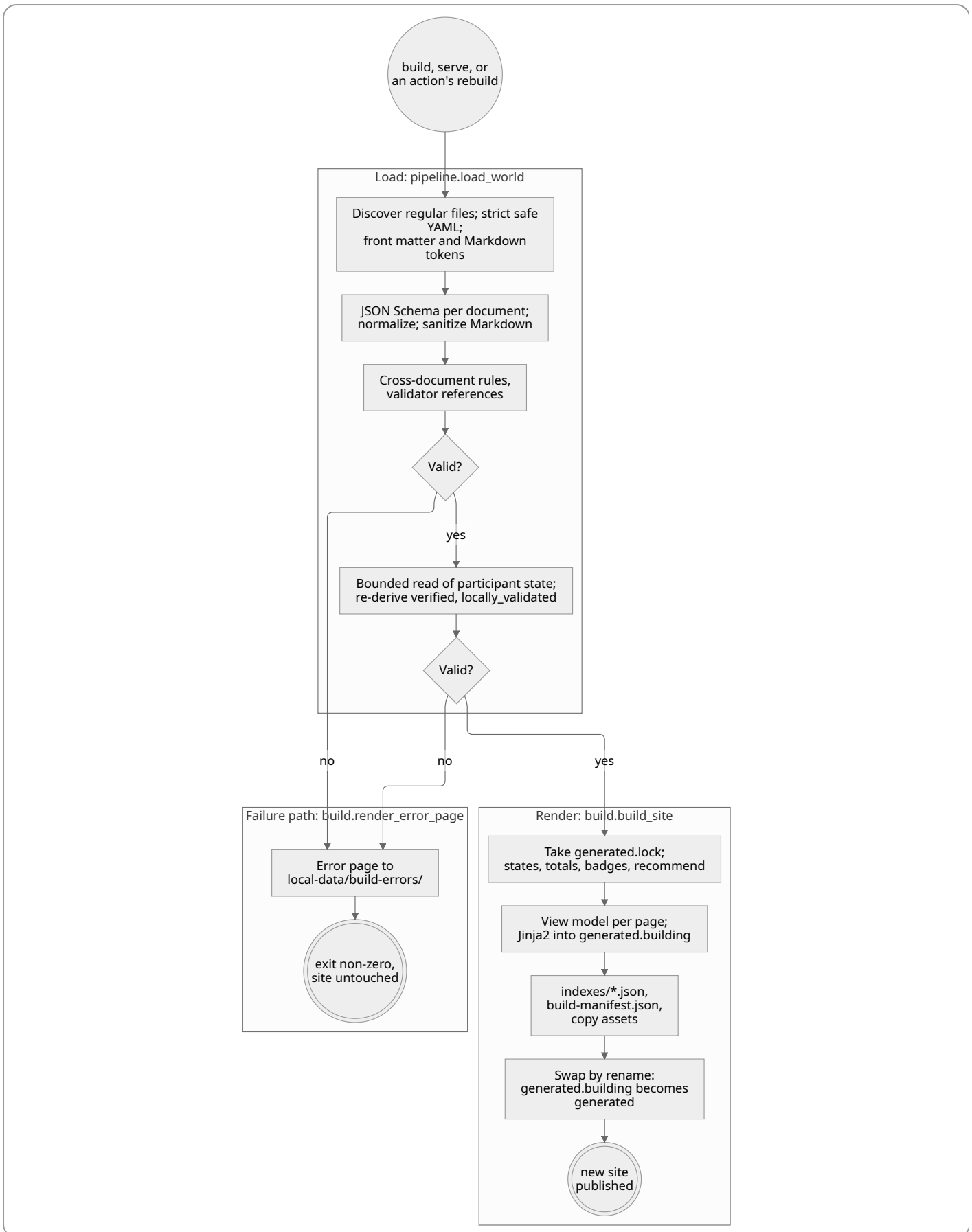


Figure 10. The build pipeline (`pipeline.load_world` , `content_loader.load_content` , `build.build_site` , `build._swap` , `build.render_error_page`).

Open issues in this flow (round 12). Validation result files and `review*.yaml` files are read without the size bound that protects `progress.yaml` , so a FIFO or a link to a device can hang or kill a build (E2, E3, High).

Evidence files have no size ceiling, so a very large log slows every build (E8, Medium).

4.7 Taking an upstream curriculum update

Walkthrough.

1. The participant runs the preflight. It checks: a Git repository; no merge in progress (checked first, because the working-tree advice would otherwise be to commit conflict markers); a clean working tree; an `upstream` remote; the participant folder. Each is `pass`, `warning` or `fail`.
2. It reports what a migration would do and which attempts are on an older quest version than upstream now publishes. Those attempts are not moved.
3. If anything fails, it prints the fix for each and exits non-zero. Otherwise it prints a backup branch name and the commands, and stops. It runs no merge and creates no branch.
4. The participant runs the printed commands: create the backup branch, fetch, read the incoming log, merge. Because curriculum and participant work are in separate folders, the merge touches only program-owned files unless both sides edited the same file.
5. The participant runs `make validate-content`. If it reports that the progress schema is old, they run `make migrate`.
6. The migration refuses during a merge, takes the progress lock, reads the file with a bounded read, applies each step (never advancing an in-progress attempt's quest version, never dropping an unknown field), validates, writes atomically, reloads the whole world, and restores the original bytes if the reload fails.
7. If the merge goes wrong: `git merge --abort`, or reset to the backup branch.

BASH

```
quest-app update
make update-check
make migrate
```

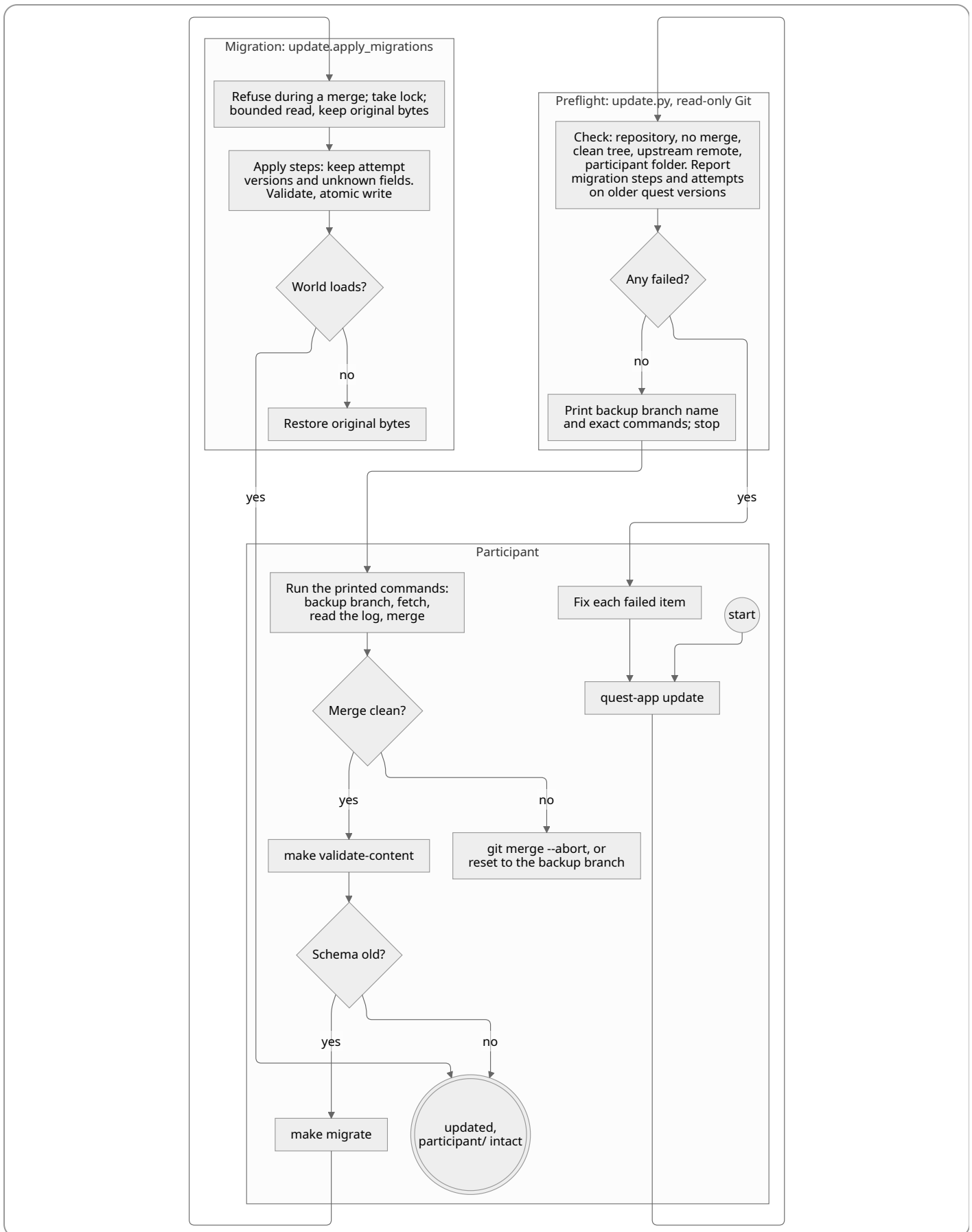


Figure 11. Taking an upstream update (`update.preflight` , `update.update_instructions` , `update.apply_migrations` , `migrations.migrate`). The flow starts at `start` in the Participant lane.

Open issues in this flow (round 12). A `locally_validated` attempt becomes a load error after an update that adds a validator to its quest, and every action is then refused (E5, High). `make migrate` rewrites `progress.yaml`

without an activity line (E10, Low).

4.8 One browser action through the loopback service

Walkthrough. The participant presses a button on a served page, for example Submit on the quest page. The page is an ordinary HTML form; no JavaScript is needed.

1. The browser posts `application/x-www-form-urlencoded` to `/api/action/submit-for-review/<quest-id>`. The form carries the per-run token, which the service substituted into the page when it served it, and the confirmation checkbox.
2. **Host check.** A `Host` header that is not a loopback name is refused, which defeats DNS rebinding (ADR-041).
3. **Origin check.** If `Origin` or `Referer` is present, it must be `http` on a loopback name at the bound port.
4. **Body.** The content type must be the form type and the declared length at most 64 KiB, checked before the body is read.
5. **Token.** Compared in constant time with the token in memory.
6. **Allowlist.** The action must be in `MUTATING_ACTIONS`; the path carries only IDs.
7. **Confirmation.** A confirmed action without the checkbox is sent back to the page with the confirmation text. `ActionRunner.perform` checks the same rule again, so the JSON endpoint and the CLI meet it too (ADR-033).
8. **Locks.** The service lock serializes requests in this process; the progress lock serializes against a CLI action in another process.
9. **Load, guards, write.** The action layer loads the world, applies the action's guards, and writes records atomically.
10. **Rebuild.** The site is rebuilt with live controls, because the service is running.
11. The service answers `303 See Other` back to the page, with a flash message for a refusal or an advisory. Every failure path answers with a status and a message; none escapes as a traceback.

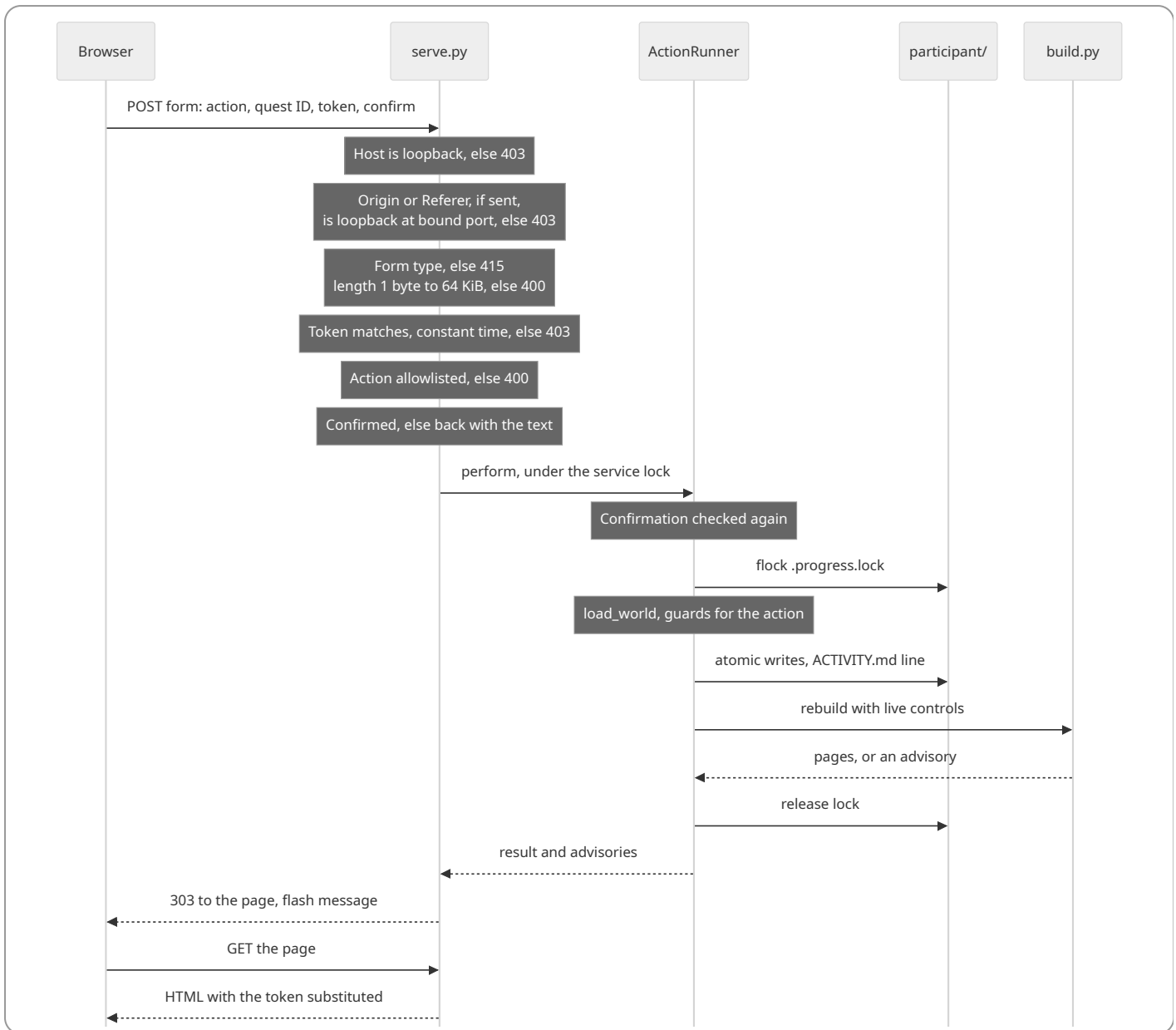


Figure 12. A form action through the loopback service (`serve.Handler._post`, `_handle_form_action`, `_host_is_acceptable`, `_origin_is_acceptable`, `_read_form_body`, `_token_matches`, `ActionRunner.perform`).

Open issue: this flow does not work in a real browser at 16a0b03. Round 12 finding C1 (Blocking): every page is served with `Referrer-Policy: no-referrer`, so Chromium sends `Origin: null` on a form post. `_origin_is_acceptable` parses `null` as a non-`http` origin and refuses the request as cross-origin at step 3. Only the CLI can change state at this commit. No test submitted a form end to end. A fix is in progress on `fix/r12-web` and is not merged.

Part 5. The security model

A local application still has attackers. Every program on the machine can reach a loopback port, every web page the participant opens can try to post to it, and the repository it reads can arrive from someone else as a pull request. This part states the threats and the control that answers each, with the code that implements it.

5.1 Threats

From `docs/SECURITY-AND-PRIVACY.md`, restated as what the application must prevent:

1. A web page in the participant's browser making the local service change state (cross-site request forgery, DNS rebinding).
2. Any caller running an arbitrary command or reading or writing an arbitrary path through the service or the CLI.
3. Malicious Markdown in curriculum or evidence running script in the generated pages.
4. A validator reading credentials, escaping its folders, hanging, or flooding output.
5. A secret reaching evidence, a log, a validation result or Git.
6. A forged `verified` state, or an approval of evidence the reviewer never saw.
7. A hostile repository (a participant's fork checked out by a reviewer) crashing or hanging the application through symbolic links, FIFOs, huge files or deeply nested YAML.

5.2 The loopback boundary

The service is the only component that accepts requests, so it carries most of the controls. A request meets them in this order (`quest_app/serve.py`):

Control	Rule	Why
Loopback bind	<code>assert_loopback</code> refuses any bind address other than <code>127.0.0.1</code> , <code>::1</code> or <code>localhost</code> at startup	Nothing off the machine can connect
Host check	A request whose <code>Host</code> is not a loopback name gets 403 before anything is rendered (ADR-041)	A page at an attacker's domain that resolves to <code>127.0.0.1</code> is same-origin to the browser and could read a served page and take the token from it
No body on GET	A GET with a declared body is refused	An undeclared body on a kept-alive connection would be parsed as a second request with attacker-chosen headers
Origin check	For state-changing requests, <code>Origin</code> and <code>Referer</code> , when present, must be <code>http</code> on a loopback name at the bound port	A page on another site cannot post here. A missing header is accepted because a command-line client sends none; the token is what authorizes
Strict body	JSON or form content type only, declared length at most 64 KiB, checked before reading	Bounds memory and parsing
Per-run token	Minted at startup, held in memory, substituted into served HTML, compared in constant time; never written to disk or printed	Only a page this run served can act. A page opened from disk has only the placeholder

Control	Rule	Why
Action allowlist	The action must be in <code>MUTATING_ACTIONS</code> ; the quest ID must exist in loaded content; a validator must be declared by the quest	No path and no command crosses the boundary: the caller names IDs, the server resolves them
Confirmation	<code>start-quest</code> , <code>submit-for-review</code> and <code>record-review</code> need an explicit confirmation, checked in the action layer for every caller (ADR-033)	The actions that create files, hand work to someone else, or produce verified XP happen only when a person said so in that request
Response headers	Content Security Policy <code>default-src 'none'; script-src 'self'; style-src 'self'; img-src 'self' data:; form-action 'self'; base-uri 'none'; frame-ancestors 'none'</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: no-referrer</code> , <code>Cache-Control: no-store</code>	No remote script, style or frame; no framing by another page
Catch-all	Both handlers answer every exception with a status and the exception type, never a traceback	A traceback would print absolute paths and leave the client with no answer after a change had landed

Open (round 12, C1, Blocking). The `Referrer-Policy: no-referrer` header and the origin check conflict: Chromium sends `Origin: null` on a form post from such a page, and the check refuses it. At `16a0b03` every browser form is refused and only the CLI changes state. The fix is in progress on `fix/r12-web`.

5.3 No arbitrary command, no arbitrary path

- **Commands.** There is no code path from a request to a shell. Validators are Python callables imported by an allowlisted module path from the `validators` package, started in a child with a fixed argument list. Validator parameters are typed enumerations or numbers; there is no string parameter type. `git_status.py` runs exactly five read-only Git commands and raises if asked for another; `update.py` runs three read-only ones (ADR-014, ADR-032).
- **Paths.** Participant paths in records keep the contract prefix `participant/` and are resolved by `AppConfig.resolve_participant_path`, which rejects absolute paths, `..` segments and null bytes, resolves symbolic links, and re-checks that the result is inside the participant root (ADR-018). A validator's `workspace` applies the same rule to its registered roots. The service resolves static file paths the same way under `generated/`.
- **Deletion.** `tools/clean.py` (behind `make clean`) removes an exact allowlist of machine-owned paths and refuses anything reached through a symbolic link.

5.4 Allowlisted writes

The application writes to these places only:

Target	Writer	Notes
<code>participant/progress.yaml</code>	<code>store.ProgressStore.write</code>	Schema-validated before writing, atomic, under the progress lock
<code>participant/ACTIVITY.md</code>	<code>store.append_audit</code>	One line per change
<code>participant/evidence/<quest>/<attempt>/</code>	<code>store</code> , <code>evidence.store_result</code> , <code>review</code>	Template files on start (never overwriting), validation results, <code>submission.yaml</code> , <code>review.yaml</code> and archives

Target	Writer	Notes
<code>generated/</code> , <code>generated.lock</code> , <code>generated.building/</code> , <code>generated.previous/</code>	<code>build.py</code>	Disposable
<code>local-data/build-errors/</code> , <code>local-data/service-ports/</code>	<code>build.py</code> , <code>serve.py</code>	Disposable
A validator's registered write roots	The validator, through <code>Workspace.write_text</code>	None of the shipped validators declares a write root

Open (round 12, E1, Blocking). Three writes do not yet meet this rule. The activity line is appended with a plain `open("a")`, which follows a symbolic link out of `participant/`, and a FIFO named `ACTIVITY.md` hangs the action while it holds both locks. Validation results are written into a `validation/` directory that can itself be a symbolic link leading outside the package. The review archive is written with a plain `write_text`. Fixes are in progress on `fix/r12-io`.

5.5 Untrusted content in the pages

- Quest Markdown is rendered with raw HTML disabled, then sanitized by `nh3` with an explicit tag, attribute and URL-scheme allowlist (ADR-021).
- Jinja2 autoescapes every template value.
- `PROOF.md` is rendered through the same path. A symbolic link inside the package that resolves outside it is not rendered.
- Validator output is shown as text, never as HTML.
- The search index holds published quest text only, never evidence.
- Pages load no remote script, font, image or analytics, and the CSP forbids them.

5.6 Validator containment

Section 2.6 lists the controls: constructed environment, repository-first import path, own process group, timeout that kills the group, 1 MiB stream ceiling, output limit, redaction of output and of every check field, schema validation of the result. They bound what a validator does through the `Workspace` and the process. They are **not** an operating-system sandbox: nothing blocks a socket, a direct `open()` bypasses the roots, and a process that starts its own session escapes the group. Container or seccomp isolation is deferred (D9). Validators are program-owned and reviewed like application code.

5.7 Secret scan and redaction

- **The detectors.** `quest_app/secret_patterns.py` holds one set of patterns (AWS keys, GitHub, Slack, Google, Anthropic, OpenAI, Atlassian, Stripe and npm tokens, JSON web tokens, private-key blocks, bearer headers, credentials in URLs, and keyword assignments such as a `password` or `secret` field with a value). Every scan and every redaction uses it.
- **Repository hygiene.** `tools/secret_scan.py` (`make secret-scan`, part of `make check`) scans every tracked text file and prints `path:line:column` and the pattern, never the value. A line ending in an allow pragma is skipped; this is how the scanner's own test fixtures live in the repository.
- **The evidence gate.** `evidence.scan_evidence` scans the whole evidence package before `mark-evidence-ready` and again before submission. It calls the detectors directly, so a participant cannot switch the check off with the pragma. A file it cannot read, and a link leading outside the package, count as findings: the scan

cannot clear what it may not read. Images and archives are skipped, which is why a screenshot is not scanned (a known limitation).

- **Redaction.** Validator output and every free-text check field pass through `redact_text` before they are stored or shown. Error messages replace path-shaped text with `<path>`.

Why a gate and not a report. A false positive costs the participant a minute. A missed credential costs a rotation. The scan is a safety net, and the documents say so: widening the detectors is deferred until there is a corpus to measure false positives against (D10).

Open (round 12, E7, Medium). Redaction runs after truncation, so a token cut at the truncation boundary can be stored in clear.

5.8 Review integrity

- `verified` is re-derived from the review record on every load; an attempt that claims it without a matching approval is refused as an integrity error (Part 3, Section 3.5).
- An approval needs a verification statement; a request for changes needs a finding.
- An approval of evidence whose hash or proof-file digests changed since submission is refused unless the reviewer acknowledges it; after approval, a later change raises a warning that the approval may be stale.
- Reviewer identity is a display name plus Git history, not a signature (ADR-030). The limitation is stated in the reviewer guide and the interface.

Open (round 12, E4, Medium). A hand-edited `state: submitted` with no `submission.yaml` loads and can be approved, which skips the secret-scan gate that submission applies.

5.9 Hostile repository files

A reviewer runs the application against a participant's repository, so participant files are treated as hostile input:

- content discovery reads only regular files inside the content tree (a link to `/dev/zero` or a FIFO is refused);
- `progress.yaml`, `submission.yaml` and `review.yaml` are read through `safe_io.read_bounded_bytes`: an ordinary file under 2,000,000 bytes, checked with `stat` before reading;
- YAML is parsed with `StrictSafeLoader`: safe constructors, duplicate keys refused, deep nesting reported (ADR-025, ADR-029);
- the service-port files are read with a 16-byte bound.

Open (round 12, E2 and E3, High). Validation result files and `review*.yaml` files other than `review.yaml` are read without that bound; a FIFO hangs `validate`, `build` and the service, and a stray unparsable `reviewer-notes.yaml` crashes `build`. Fixes are in progress on `fix/r12-io`.

5.10 External systems and privacy

Release one performs **no external write** and makes **no outbound request** of its own. Every quest declares `risk.external_write: false`, every validator is registered `network: denied`, and `tests/test_release_facts.py` fails the day a quest declares a write. The preview-and-confirm requirements in `docs/SECURITY-AND-PRIVACY.md` are the conditions a write-capable quest must meet before it is added, not a mechanism that runs today. No telemetry leaves the machine.

Part 6. Key decisions

`docs/DECISIONS.md` is the full log: forty-one architecture decision records (**ADRs**), appended rather than rewritten, several amended after an audit found the code and the decision disagreeing. This part groups them by the question each one answers and gives the reason in one or two sentences. Read the log for the evidence behind each.

6.1 Product shape

ADR	Decision	Why
001	The quest is its own repository	Participants fork, run and update it independently of any other program
002	Participants fork the canonical repository	A fork keeps upstream history, so curriculum updates arrive by merge
003	Three ownership zones in separate top-level folders	Updates cannot overwrite participant work; generated output stays disposable
010	A restrained field-guide visual direction	Motivation without looking childish in an enterprise setting
011	Only a reviewer decision produces verified progress	Automated checks establish facts, not the quality or meaning of the work
013	Repository evidence is primary; screenshots support	Files, tests and logs are reproducible and diffable

6.2 Content and contracts

ADR	Decision	Why
004	Markdown with YAML front matter for quests, YAML for structured content	Readable diffs and approachable authoring
005	JSON Schema is the public contract	Language-neutral, testable, editor-friendly
012	Stable IDs; attempts record quest version and content hash	Titles and filenames change; version-aware attempts survive curriculum edits
016, 026	Acceptance criteria are parsed from the body with positional IDs <code>ac-<n></code> ; a bullet list is a warning, an empty item or split list an error	One copy of the criteria; the failures that actually move an ID are the ones that stop the build
020	<code>jsonschema</code> against the published schemas, frozen dataclasses inside	No second rule set that can drift from the schema a maintainer reads
021	<code>markdown-it-py</code> with raw HTML off, then <code>nh3</code> sanitization	The guarantee does not depend on the renderer's escaping
025, 029	YAML through a strict safe loader: no custom constructors, no duplicate keys, bounded nesting	<code>xp: 10</code> then <code>xp: 90</code> must not mean one thing in review and another at build
027	Markdown structure from the token stream, never from lines	Line regexes turned code blocks and nested items into criteria

ADR	Decision	Why
028	A content-hash mismatch is a warning, never fatal	A typo fix must not revoke approved work, but the reader must be told

6.3 Generation and runtime

ADR	Decision	Why
006	Python and Jinja2 generate the site deterministically	Server-rendered HTML is inspectable and needs no frontend build
007	No frontend framework	A framework would duplicate state the Python model already holds
008	Static generation plus a loopback service for the few mutating actions	Static pages cannot write files; a full web app is unnecessary
009	Mermaid documents; HTML and CSS define the UI	Diagrams describe relationships, not layout or accessibility
015	Generated output is not canonical	A build can always recreate it
022	The service uses the standard library <code>http.server</code>	Every requirement is a restriction; a framework hides the surface they apply to
019	Python 3.10 or newer (amended from 3.12)	The sandbox a participant may be given ships 3.10
023, 024	pytest, Playwright, axe-core; ruff and <code>mypy --strict</code>	One toolchain locally and in CI
037	The service records each bound port under <code>local-data/service-ports/</code>	A CLI action must know whether a service is running to build live or offline pages
040	The build honours <code>SOURCE_DATE_EPOCH</code>	Makes "two builds are byte-identical" true on demand

6.4 Safety and integrity

ADR	Decision	Why
014	Only registered validators and named actions run	Arbitrary command execution on localhost is an avoidable boundary failure
017	A validation run never changes attempt state; <code>locally_validated</code> is requested and guarded	A validator must not be an authority over participant state
018	Participant paths keep the <code>participant/</code> prefix; the root is configuration	Tests use fixtures without touching live work; the contract does not move
030	Reviewer provenance is conventional, and said so	A half-built signature scheme invites trust it has not earned
031	The evidence hash covers the work, not the records; proof files outside the package are digested too	The application's bookkeeping must not read as a change to the work
032	The application never pushes, opens a pull request or merges	Those are claims the participant makes, not the tool

ADR	Decision	Why
033	Every rule the browser shows is enforced in the action layer	A second caller turned a greyed-out button into a hole
039	A validator judges the attempt it was given	Records connect by ID, never by modification time
041	The service answers only to a loopback name, and every request gets a response	Defeats DNS rebinding; no traceback, no silent drop

6.5 Concurrency and failure

ADR	Decision	Why
034	One mutation at a time across processes: a file lock around load, decide and write	The CLI and the service could interleave a read-modify-write
035	One build at a time per output directory	Two builds shared one staging directory and published partial sites
036	Not being able to lock is never a reason to refuse the work	The atomic rename keeps files whole; refusing to record work is worse
038	A failed rebuild after an action is an advisory, not a failed action	The participant's record cannot be regenerated; the site can

6.6 Patterns across the decisions

Three patterns recur and explain most of the code's shape:

1. **One place per rule.** Confirmations live in `state_machine.CONFIRMATIONS`, guards in `actions.py`, secret patterns in `secret_patterns.py`, schemas in `schemas/`. A second copy is a second thing to drift, and audits repeatedly found drift where a rule had two homes.
2. **Re-derive, never believe.** Anything a participant is not allowed to decide is computed from records on every load rather than read from their file.
3. **Degrade toward the participant's record.** When something non-essential fails (a lock, a rebuild, an activity line), the participant's work is kept and the failure is reported, never the other way round.

Part 7. Status and remaining work

This part says what is finished, what is open, and what is deliberately left for later. It describes `main` at commit `16a0b03` (the merge of pull request 8, round 11), with the round 12 findings taken from `docs/audits/round-12-independent-audit.md` on the branch `chore/round-12-audit` at `7a8cf7b`.

7.1 How the work was staged

`docs/IMPLEMENTATION-PLAN.md` divides the build into eleven stages. Each ends with an audit by a fresh reviewer, recorded under `docs/audits/`, and a stage is checked off only when its audit passes after fixes.

Stage	Scope	State
0	Planning and feasibility audit	Complete
1	Repository and engineering foundation	Complete
2	Content contracts and loading	Complete
3	Deterministic site generation	Complete
4	Participant state and loopback service	Complete
5	Evidence workspaces and validator framework	Complete
6	Participant UI and accessibility	Complete
7	Submission and reviewer integrity	Complete
8	Fork updates, versioning and migrations	Complete
9	Documentation, hardening and release candidate	Complete
10	Final independent audit and release decision	Open

7.2 Why Stage 10 is still open: DH7

Stage 10 closes when every release acceptance criterion in `docs/ACCEPTANCE-CRITERIA.md` is met. Forty-seven are checked. One is open:

DH7: the final audit reports no unresolved blocking or high findings.

The project reads DH7 strictly: it closes only on an audit round that **finds nothing** blocking or high. Fixing a round's findings does not close it, because the fixes are themselves unaudited; the next round runs against the merge commit that contains them.

Method of a round. Three independent review **lenses**, each in its own worktree at the commit under audit with `docs/audits/` removed, each given only paths, a commit and areas to probe, none shown the author's beliefs or another lens's findings. The lenses cover the curriculum followed literally, the engine, and tooling, tests and documentation. Every finding is verified by reproduction or by reading the cited code before it is accepted.

7.3 Audit history

Round	Commit	Blocking	High	Record
1 (final audit)	<code>1418b78</code>	3	4	<code>stage-10-final-audit.md</code>
2 (re-audit)	<code>128617e</code>	1	3	<code>stage-10-final-re-audit.md</code>
3 (second re-audit)	<code>aafce28</code>	1	2	<code>stage-10-final-re-audit.md</code>
External review	<code>a69a8b8</code>	n/a	n/a	<code>external-review-2026-09-17.md</code> : engine a release candidate, curriculum too thin
4	<code>207a7b3</code>	0	10	<code>round-04-independent-audit.md</code>
5	<code>ff0c63d</code>	0	9	<code>round-05-independent-audit.md</code>
6	<code>ad629a2</code>	2	6	<code>round-06-independent-audit.md</code>
7	<code>31a74cb</code>	1	8	<code>round-07-independent-audit.md</code>
8	<code>b0667c0</code>	0	7	<code>round-08-independent-audit.md</code>
9	<code>5248500</code>	0	8	<code>round-09-independent-audit.md</code>
10	<code>8df0ac5</code>	1	8	<code>round-10-independent-audit.md</code>
11	<code>ea36037</code>	0	3	<code>round-11-independent-audit.md</code>
12	<code>16a0b03</code>	2	4	<code>round-12-independent-audit.md</code> (on <code>chore/round-12-audit</code>)

Every accepted blocking and high finding from rounds 1 to 11 is fixed on `main`, each code fix with a test that fails without it. The round records say which lower findings were deferred or stated as limitations instead. The pattern the audits name is consistent: defects live in states no fixture occupied (no participant yet, a refused action, a second process, a hostile file, a real browser rather than a test client).

7.4 Round 12: open findings

Round 12 audited `16a0b03` and found two blocking and four high findings, so it does not close DH7. **None of the fixes is merged.** They are in progress on four branches: `fix/r12-web`, `fix/r12-io`, `fix/r12-integrity` and `fix/r12-runner`. Until they merge and a further round runs against the merge commit, the statements below describe the code as it is.

ID	Severity	Finding
C1	Blocking	Every browser action form fails: pages send <code>Referrer-Policy: no-referrer</code> , Chromium then sends <code>Origin: null</code> , and the origin check refuses it. Only the CLI changes state
E1	Blocking	Three writes can follow a symbolic link out of <code>participant/</code> : the activity line, validation results in a linked <code>validation/</code> directory, and the review archive. A FIFO <code>ACTIVITY.md</code> hangs an action while it holds both locks
C2	High	CLI tests with <code>--participant-root</code> still build into the repository's own <code>generated/</code> , so <code>make check</code> replaces a participant's site with fixture data until the next build
E2	High	Validation result files are read without a size or file-type bound; a FIFO hangs <code>validate</code> , <code>build</code> and the service
E3	High	<code>review*.yaml</code> files are globbed and parsed without a bound or error handling; a stray bad <code>reviewer-notes.yaml</code> passes <code>validate</code> and crashes <code>build</code>
E5	High	A legitimate <code>locally_validated</code> attempt becomes a load error after an update that adds a validator to the quest, and every action is then refused
C3	Medium	A link leading outside the evidence package makes the review page say a secret was found
C4	Medium	After a pass, local validation and a failing re-run, the evidence page shows "passed" beside "Last run: fail" with nothing connecting them
C5	Medium	When a newer quest version renames a proof path, the review checklist marks the old item "Not detected" with no note that the version changed
E4	Medium	A hand-edited <code>state: submitted</code> with no <code>submission.yaml</code> loads and can be approved, skipping the secret-scan gate
E6	Medium	The runner caps the output excerpt and check count but not other check fields, so an oversize field discards the whole run
E7	Medium	Redaction runs after truncation, so a token cut at the boundary is stored in clear
E8	Medium	Evidence files have no size ceiling; a very large log makes every build slow while it holds the locks
T1	Medium	The documentation command test covers five documents and skips four guides
E9	Low	A validator grandchild that starts its own session survives cleanup
E10	Low	<code>make migrate</code> rewrites <code>progress.yaml</code> without an activity line
E11	Low	A validator that finishes but leaves a non-daemon thread running is reported <code>interrupted</code>
T2	Low	<code>PYTHON ?= python3</code> in the Makefile is never used

What this means for a reader of this document. Parts 2 to 5 describe the design and the code at `16a0b03`. Where a round 12 finding shows the code falling short of the design, the relevant part says so next to the claim.

7.5 Known limitations

These are deliberate and documented in `docs/RELEASE-NOTES.md`. They are not defects to be fixed before release; they are boundaries of release one.

1. Recording a reviewer decision needs the application (the local service or the CLI), because it writes files. Reading evidence works from generated pages alone.

2. Reviewer provenance is conventional, not cryptographic (ADR-030).
3. Environment Health reports build-time facts, not live ones (D7).
4. Reviewer-awarded badges cannot be awarded: there is no badge-award record type, so they show as pending.
5. Validator isolation is policy plus process boundaries, not a container or seccomp (D9).
6. Catalog filtering needs JavaScript; region and tag pages are the scripting-free routes.
7. No coverage reporting (D2) and no glossary content type (D1).
8. A screenshot is not scanned for secrets.
9. Clean-clone installation is tested in CI from a `git archive` export, one step short of a real `git clone`; a real clone was run by hand in round 4.
10. Earlier quest versions are not kept. An attempt records its version, but pages show the current version's criteria; Git history holds the earlier text.
11. Two builds are byte-identical only when `SOURCE_DATE_EPOCH` is set.

7.6 Deferred work

From the deferred-work register in `docs/IMPLEMENTATION-PLAN.md`. Items D3 to D6 were delivered in their stages.

ID	Work	Why deferred
D1	Glossary content type	No schema, sample or screen needs it yet
D2	Coverage reporting	Half-configured is worse than absent
D7	Live Environment Health checks through the service	A generated page cannot inspect the machine when read
D8	A fixture exercising all eight quest states at once	Tests reach the other states by mutation today
D9	Container or seccomp isolation for validators	Release one is policy plus process boundaries
D10	Wider secret-scanner coverage (distant assignments, bare keys, base64, webhooks, personal data)	Widening without a corpus risks false positives that teach people to route around the gate
D11	Role separation between participant and reviewer	The reviewer page is in the primary navigation; ADR-030 already states provenance is conventional
D12	A degraded view for an attempt whose evidence directory is missing	Today one deleted folder makes the site unbuildable until <code>progress.yaml</code> is edited

Curriculum. Release one ships eight quests, one per region, four badges and one track. `docs/CURRICULUM-BACKLOG.md` lists about 128 proposed quests across the eight regions plus seven optional "trap" challenges, from repository safety through Jira, Trello and GitHub synchronization to Playwright test repair. The backlog is a plan, not a promise, and any quest that performs an external write must first meet the preview-and-confirm conditions in `docs/SECURITY-AND-PRIVACY.md`. Several future-facing application capabilities are also named in the specification and not built: a central opt-in scoreboard, AI-assisted advisory evaluation, and external-write previews.

7.7 What done looks like

Stage 10 completes, and the project can make its release decision, when:

1. the round 12 fixes are merged, each with a test that fails without it;

2. a round runs against that merge commit and finds no blocking or high finding;
3. DH7 is checked, `docs/audits/final-audit.md` records the decision (`release` , `release-with-advisories` or `do-not-release`), and the Stage 10 boxes in `docs/IMPLEMENTATION-PLAN.md` are checked.

This document then gets a minor version for the new status (see `REGENERATING.md`).

Part 8. Glossary

Terms are defined where they first appear; this list collects them. Code names are given where the term maps to one.

Term	Meaning
Acceptance criterion	One item of the ordered list under a quest's <code>## Acceptance criteria</code> heading. Identified by position as <code>ac-<n></code> with a hash of its text (ADR-016, ADR-026)
Action	One of the ten named operations that change state: <code>start-quest</code> , <code>mark-evidence-ready</code> , <code>mark-locally-validated</code> , <code>reopen-evidence</code> , <code>submit-for-review</code> , <code>withdraw-submission</code> , <code>resume-quest</code> , <code>run-validator</code> , <code>record-review</code> , <code>rebuild</code> (<code>actions.MUTATING_ACTIONS</code>)
Action layer	<code>quest_app/actions.py</code> , specifically <code>ActionRunner</code> : the one place every action's rules are enforced, called by both the service and the CLI (ADR-033)
Activity log	<code>participant/ACTIVITY.md</code> , one line per change the application made to participant files
ADR	Architecture decision record, numbered in <code>docs/DECISIONS.md</code>
Advisory	A problem reported alongside a successful action rather than blocking it, for example an unrun validator at submission, or a rebuild that failed after a change was recorded
AI Context Engineer	The role the curriculum trains: owner of the intent and validation bookends of software delivery
Atomic write	Writing a temporary file in the same directory, flushing it to disk, and renaming it over the target, so the target is either wholly old or wholly new (<code>store.atomic_write_text</code>)
Attempt	One participant's run at one quest, recorded in <code>progress.yaml</code> with an attempt ID, quest version, content hash, state and evidence path
Attempt state	One of the six stored states: <code>in_progress</code> , <code>evidence_ready</code> , <code>locally_validated</code> , <code>submitted</code> , <code>needs_changes</code> , <code>verified</code> (<code>models.AttemptState</code>)
Available	Computed quest state: every prerequisite verified, no attempt yet. Never stored
Badge	A recognition defined in <code>content/badges/</code> , awarded automatically from verified quests, or by a reviewer or the program (reviewer awards are not implemented in release one)
Blocking / High / Medium / Low	Audit finding severities. Blocking: unsafe, data-loss risk, architecture violation, or core flow unusable. High: material requirement failure or likely confusion. Medium: quality, resilience or accessibility weakness. Low: polish
Bookend	One of the two ends of delivery the role owns: intent (requirements, work items) and validation (tests, evidence)
Build lock	Exclusive file lock on <code>generated.lock</code> held for a whole build (ADR-035)
Claimed XP	XP from quests in <code>evidence_ready</code> or any later state. The participant's own record, never added to verified XP
Confirmation	An explicit "I mean this" in the request, required for <code>start-quest</code> , <code>submit-for-review</code> and <code>record-review</code> . A checkbox in the browser, <code>--confirm</code> on the CLI, <code>"confirm": true</code> in JSON (<code>state_machine.CONFIRMATIONS</code>)
Content hash	SHA-256 over a quest's front matter and body. Recorded on an attempt at start; a later mismatch is a warning (ADR-028)

Term	Meaning
Content problem	The single error shape every layer reports: file, stable ID, field path, rule, redacted received value, suggestion (<code>errors.ContentProblem</code>)
Curriculum maintainer	The person who writes quests, regions, badges, tracks and validators. Also called curriculum author
DH7	The one open release acceptance criterion: the final audit reports no unresolved blocking or high findings. Closes only on a round that finds none
Evidence hash	SHA-256 over an evidence package, excluding <code>validation/</code> , <code>submission.yaml</code> and <code>review*.yaml</code> (ADR-031)
Evidence package	The folder <code>participant/evidence/<quest-id>/<attempt-id>/</code> holding <code>PROOF.md</code> , <code>manifest.yaml</code> , logs, screenshots, validation results, the submission and review records
Fork	The participant's copy of the canonical repository; its <code>origin</code> remote. Curriculum arrives from the <code>upstream</code> remote
Generated output	Everything under <code>generated/</code> : HTML, JSON indexes, build manifest. Disposable and never authoritative (ADR-015)
Lens	One of the independent reviewers in an audit round, each with its own scope and worktree
Loopback service	The HTTP server in <code>serve.py</code> , bound to <code>127.0.0.1</code> , that serves the pages and performs actions
Locked	Computed quest state: at least one prerequisite is not verified. Never stored
Machine-owned	The ownership zone of <code>generated/</code> and <code>local-data/</code> : disposable, ignored by Git
Migration	A step that moves <code>progress.yaml</code> from one schema version to the next, validated before and after, reversible by restoring the original bytes (<code>migrations.py</code>)
Needs changes	Attempt state after a reviewer asks for changes or rejects. The participant resumes from it
Origin check	Refusal of a state-changing request whose <code>Origin</code> or <code>Referer</code> is present and is not <code>http</code> on a loopback name at the bound port
Participant	The learner who works through quests in their own fork
Participant-owned	The ownership zone of <code>participant/</code> : the participant's files, never replaced by an update
Preflight	<code>quest-app update</code> : read-only checks before taking an upstream update. Prints commands; runs no merge
Prerequisite	A quest that must be verified before another can start
Program-owned	The ownership zone of <code>content/</code> , <code>schemas/</code> , <code>templates/</code> , <code>assets/</code> , <code>quest_app/</code> , <code>validators/</code> and most of <code>docs/</code> : changed upstream
Progress lock	Exclusive file lock on <code>participant/.progress.lock</code> held across load, decision and write of every action (ADR-034)
Progress store	<code>store.ProgressStore</code> , the only writer of <code>participant/progress.yaml</code>
PROOF.md	The participant's narrative for one attempt: what was built, where, how to reproduce, what was checked, what remains, and a sensitive-values confirmation

Term	Meaning
Proof files	Digests of every declared proof path outside the evidence package, recorded on submission and review so a change to them is detected (ADR-031, amended)
Proof requirement	An entry in a quest's <code>proof.required</code> or <code>proof.optional</code> : a file, directory, validator result, command record, screenshot, demonstration or review
Qualifying result	A validator run whose outcome is <code>pass</code> or <code>warning</code> . Only these count toward <code>locally_validated</code>
Quest	One unit of curriculum: a Markdown file with YAML front matter under <code>content/quests/<region>/</code>
Quest version	An integer the author raises when a quest's evaluable meaning changes. An attempt keeps the version it started on
Redaction	Replacing secret-like values in validator output, check fields and messages before they are stored or shown (<code>secret_patterns.redact_text</code>)
Region	A group of quests on the map, such as Base Camp or Jira Jungle. Defined in <code>content/regions/</code>
Registry	<code>validators/registry.yaml</code> : the complete list of programs the application may run, with their constraints
Request token	A random value minted when the service starts, held in memory, substituted into served pages, and required on every state-changing request
Review record	<code>review.yaml</code> in the evidence package: the reviewer's decision, findings, verification statement, evidence hash and proof files. Superseded records are archived as <code>review-<timestamp>.yaml</code>
Reviewer	The person who evaluates submitted evidence and records a decision. Also called Questmaster
Round	One release audit pass against a specific commit, recorded under <code>docs/audits/</code>
Secret scan	Detection of secret-like values: over tracked files by <code>tools/secret_scan.py</code> , and over an evidence package by <code>evidence.scan_evidence</code> , where it blocks marking evidence ready and submission
Stable ID	A lowercase, hyphenated identifier that never changes when a title or filename does, and is never reused (ADR-012)
Submission record	<code>submission.yaml</code> in the evidence package: what was submitted, with its evidence hash and proof files
Track	An ordered or rule-based selection of quests, defined in <code>content/tracks/</code>
Upstream	The canonical repository participants take curriculum updates from
Validator	A program-owned Python function, registered by ID, that checks participant artifacts and returns structured findings. Never an authority over state (ADR-017)
Validator outcome	One of <code>pass</code> , <code>fail</code> , <code>warning</code> , <code>environment_failure</code> , <code>inconclusive</code> , <code>interrupted</code>
Verification statement	The reviewer's own words, at least twenty characters, saying what they checked and how. Required to approve
Verified	Attempt state produced only by a reviewer's approval, and re-derived from the review record on every load (ADR-011)
Verified XP	XP from verified quests only
View model	A display-ready object built in <code>view_models.py</code> for one page. Templates read only view models

Term	Meaning
Workspace	The object a validator receives, whose reads and writes are confined to the registered roots after symbolic links are resolved, and which exposes the attempt's own evidence files
XP	Experience points declared per quest (<code>xp</code>), shown as two separate totals: claimed and verified

Golden Thread Quest system design, version 1.0.0. Rendered from docs/design/ by
scripts/build_design_html.py.